

BarrierNet: Differentiable Control Barrier Functions for Learning of Safe Robot Control

Wei Xiao , *Member, IEEE*, Tsun-Hsuan Wang , *Student Member, IEEE*, Ramin Hasani, Makram Chahine , Alexander Amini , *Member, IEEE*, Xiao Li , *Member, IEEE*, and Daniela Rus , *Fellow, IEEE*

Abstract—Many safety-critical applications of neural networks, such as robotic control, require safety guarantees. This article introduces a method for ensuring the safety of learned models for control using differentiable control barrier functions (dCBFs). dCBFs are end-to-end trainable and guarantee safety. They improve over classical control barrier functions (CBFs), which are usually overly conservative. Our dCBF solution relaxes the CBF definitions by: 1) using environmental dependencies; 2) embedding them into differentiable quadratic programs. **These novel safety layers are called a BarrierNet.** They can be used in conjunction with any neural network-based controller. They are trained by gradient descent. With BarrierNet, the safety constraints of a neural controller become adaptable to changing environments. We evaluate BarrierNet on the following several problems: 1) robot traffic merging; 2) robot navigation in 2-D and 3-D spaces; 3) end-to-end vision-based autonomous driving in a sim-to-real environment and in physical experiments; 4) demonstrate their effectiveness compared to state-of-the-art approaches.

Index Terms—Control barrier function (CBF), neural networks, robot learning, safety guarantees.

I. INTRODUCTION

THE deployment of learning systems in decision-critical applications such as autonomous vehicle control requires safety guarantees because one simple control mistake can lead to catastrophic outcomes. Many key aspects of the perception and navigation of robots can be expressed as state-of-the-art (SOTA) learned models. While the deep-learning methodology has brought significant advancements in the development of reliable complex systems, it is still challenging to guarantee the safety of the resulting controller. This is because SOTA models

Manuscript received 8 October 2022; revised 21 December 2022; accepted 16 February 2023. Date of publication 21 March 2023; date of current version 7 June 2023. This paper was recommended for publication by Associate Editor Rafael Murrieta-Cid and Editor Wolfram Burgard upon evaluation of the reviewers' comments. This work was supported in part by Capgemini Engineering, in part by the United States Air Force Research Laboratory and the United States Air Force Artificial Intelligence Accelerator and was accomplished under Cooperative under Grant FA8750-19-2-1000, and in part by the Office of Naval Research (ONR) under Grant N00014-18-1-2830. (Wei Xiao and Tsun-Hsuan Wang contributed equally to this work.) (Corresponding author: Wei Xiao.)

The authors are with the Computer Science and Artificial Intelligence Lab, Massachusetts Institute of Technology, Cambridge, MA 02139 USA (e-mail: weixy@mit.edu; tsunw@mit.edu; rhasani@mit.edu; chahine@mit.edu; amini@mit.edu; xiaoli@mit.edu; rus@mit.edu).

Code: <https://github.com/Weixy21/BarrierNet>

This article has supplementary material provided by the authors and color versions of one or more figures available at <https://doi.org/10.1109/TRO.2023.3249564>.

Digital Object Identifier 10.1109/TRO.2023.3249564

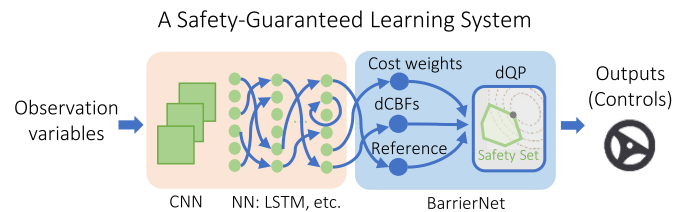


Fig. 1. Safety-guaranteed learning system with the proposed BarrierNet controller for safety-critical systems. The BarrierNet learns adaptively changing dCBFs and other parameters together with differentiable QPs (dQP), and it can guarantee safety that is enforced by dCBFs in a nonoverly-conservative way.

trained offline do not perform well online, and they have trouble with previously unseen environments and situations.

In this article, we use insights from model-based safety controllers [1], [2] to equip end-to-end learning systems with safety guarantees. We propose a new algorithm for synthesizing safe neural controllers end-to-end by defining a novel instance of control barrier functions (CBFs) that is differentiable. CBFs are popular methods for guaranteeing safety of model-based controllers when the system dynamics are known. A large body of work has studied variants of CBFs [3], [4], [5], [6], [7] and their characteristics under increasing uncertainty [8], [9], [10]. **However, as uncertainty increases, CBFs cause the system's behavior to be excessively conservative [11].**

In this article, we address the over-conservativeness of CBFs by replacing the set of hard constraints in **high-order CBFs (HOCBFs) [12]** for arbitrary-relative-degree systems with a set of differentiable constraints. We do so without loss of safety guarantees. We obtain a versatile safety guaranteed barrier layer, which we term a BarrierNet, that can be combined with any deep learning system [13], [14], [15], [16], [17]. BarrierNet can be trained end-to-end via reverse-mode automatic differentiation [18] (see Fig. 1).

BarrierNet allows the safety constraints of a neural controller to be adaptable to changing environments. A canonical application of BarrierNet is end-to-end vision-based autonomous driving (see Figs. 1 and 2). In this example, the BarrierNet is trained in a sim-to-real environment and outputs acceleration and steering commands to navigate the vehicle along the center lane while avoiding obstacles. The sim-to-real environment can ensure safety during training and allows us to directly deploy the neural controller to a real vehicle. The system and environmental observations are inputs to the upstream network, whose outputs serve as arguments to the BarrierNet layer. Finally, BarrierNet

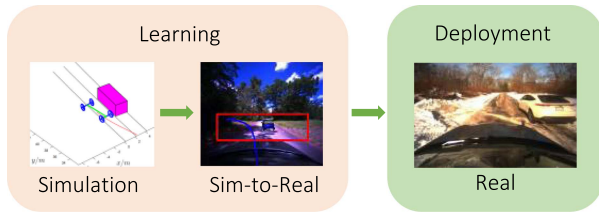


Fig. 2. Sim-to-real learning and deployment with the proposed BarrierNet controller for end-to-end vision-based autonomous driving. The model is learned in a simulator with vehicle dynamics and photo-realistic sensor synthesis and directly deployed on a full-scale autonomous vehicle.

outputs the controls that guarantee collision avoidance. In contrast to existing work, **our proposed architecture is end-to-end trainable, including the BarrierNet for neural networks.**

Contributions: This article contributed the following.

- 1) We propose BarrierNet, a novel trainable and interpretable layer that can be added to neural networks and is built by leveraging the definition of higher-order CBFs; BarrierNet provides safety guarantees for general control problems with neural network controllers, and it can be applied to general robot navigation and manipulation tasks.
- 2) We resolve the over-conservativeness of CBFs by introducing differentiable constraints in their definition (c.f., Fig. 1), which makes the CBF parameters learnable from data. This allows end-to-end training along with a given learning system.
- 3) We design BarrierNet such that the CBF parameters are adaptive to changes in environmental conditions and can be learned from data.
- 4) We train, evaluate, and verify BarrierNet on the following several robot-centric problems: a) traffic merging for autonomous vehicles; b) robot navigation with obstacles in 2-D and 3-D; c) end-to-end vision-based autonomous driving; we also deploy and evaluate the learned BarrierNet model on a full-scale autonomous vehicle for safe lane following and obstacle-avoidance with physical parked vehicles and augmented reality (AR) dynamic obstacles.

This rest of this article is organized as follows. In Sections II and III, we provide the related work and the necessary background to construct our theory. We formulate the problem in Section IV and introduce BarrierNets in Section V. Section VI includes our experimental evaluation on simple examples. The proposed BarrierNet is extended to learning for vision-based end-to-end autonomous driving in Section VII, with real car experiments given in Section VIII. Finally, Section IX concludes this article.

II. RELATED WORK

This article builds on a large body of work in machine learning, safe planning, optimization, model-based control, and CBFs.

A. Safe Planning and Control

Planning and control methods, such as the Hamiltonian analysis [19], model predictive control [20], theorem proving [21],

reachability analysis [22], [23], rapidly-exploring randomized trees [24], A^* [25], and reactive planning [26], are widely used in safety-critical systems. Conformance checking [27] can be used to bridge the gap between simulations and real systems, while sensor failures can be addressed using reachability analysis [23]. Compared to CBFs, these planning and control methods are usually computationally expensive. Further, these planning methods usually require many hand-tuned parameters and much environmental information. By comparison, the BarrierNet method described in this article learns parameters and only requires environment observations (such as front-view images). Moreover, BarrierNet can be combined as an additional learnable layer with any learning system and as a higher-frequency safety control module with any planning methods.

1) *CBF and Learning:* A large body of work studied CBF-based safety guarantees [3], [4], [5], [6], [7] and **their characteristics under increasing model uncertainty** [8], [9], [10]. Many existing works [2], [3], [28] combine CBFs for systems with quadratic costs to form optimization problems. Time is discretized. An optimization problem with constraints given by the CBFs [inequalities of the form (4) is solved at each time step. The inter-sampling effect is considered in [28] and [29]. **Replacing CBFs by HOCBFs allows us to handle constraints with arbitrary relative degrees** [12]. In this prior work, as the uncertainty of models increases, CBFs make the system's behavior excessively conservative [11]. Feasibility guarantees of the CBF method under control input constraints have been studied in [30] and [31]. However, these approaches did not consider the conservativeness of the CBF method.

The recently proposed adaptive CBFs (AdaCBFs) [32] addressed the conservativeness of the HOCBF method by multiplying the class $\mathcal{K}$ functions of an HOCBF with some penalty functions. These penalty functions are HOCBFs and are guaranteed to be nonnegative. This is due to the fact that the main conservativeness of the HOCBF method comes from the class $\mathcal{K}$ functions. By multiplying (relaxing) the class $\mathcal{K}$ functions with some penalty functions, **it has been shown that the satisfaction of the AdaCBF constraint is a necessary and sufficient condition for the satisfaction of the original safety constraint $b(x) \geq 0$** [32]. This is conditioned on designing proper auxiliary dynamics for all the penalty functions based on task specifics. However, the design of such auxiliary dynamics remains a challenge, which we study in this article. **In other words, we use the training data to determine the variation of the penalty functions in the proposed BarrierNet.**

Supervised learning techniques have been proposed to learn safe set definitions from demonstrations [33] and sensor data [34], which are then enforced by CBFs. The authors in [5] used data to learn system dynamics for CBFs. In a similar setting, [35] used adaptive control approaches to estimate the unknown system parameters in CBFs. In [36], neural network controllers are trained using CBFs in the presence of disturbances. Learning involving uncertainties with CBFs has been studied in [5] and [6], which could enable robust control for safety guarantees. These prior works address learning safe sets and dynamics, whereas we focus on the design of environment-dependent and trainable CBFs, and we can directly apply these learning methods to the proposed BarrierNet.

2) *Optimization-Based Safety Frameworks*: Recent advances in differentiable optimization methods show promising directions for safety guaranteed neural network controllers [37], [38], [39], [40], [41], [42]. In [38], a differentiable quadratic program (QP) layer, called OptNet (filter for safe control [37]) was introduced. In [43], [44], [45], and [46], safety guaranteed neural network controllers have been learned through verification-in-the-loop training. By comparison, BarrierNet is computationally efficient, easily scalable, general for control problems, and adaptive to environmental changes.

III. BACKGROUND

In this section, we briefly introduce CBFs and refer interested readers to [1] for more details. Intuitively, CBFs are a means to translate state constraints to control constraints under affine dynamics. The controls that satisfy those constraints can be efficiently solved by formulating a QP. We start with the definition of class $\mathcal{K}$ functions.

Definition 1. (Class $\mathcal{K}$ function [47]): A continuous function $\alpha : [0, a) \rightarrow [0, \infty)$, $a > 0$ is said to belong to class $\mathcal{K}$ if it is strictly increasing and $\alpha(0) = 0$. A continuous function $\beta : \mathbb{R} \rightarrow \mathbb{R}$ is said to belong to the extended class $\mathcal{K}$ if it is strictly increasing and $\beta(0) = 0$.

Consider an affine control system of the form

$$\dot{\mathbf{x}} = f(\mathbf{x}) + g(\mathbf{x})\mathbf{u} \quad (1)$$

where $\mathbf{x} \in \mathbb{R}^n$, $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$, and $g : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times q}$ are locally Lipschitz, and $\mathbf{u} \in U \subset \mathbb{R}^q$, where U denotes a control constraint set. Each of the control component bounds is assumed to be independent of each other (since we consider low-speed experiments in this work, such as autonomous driving), but the CBF-based method can still work when control bounds are coupled, which can be written as different forms of input constraints in the QP.

Definition 2: A set $C \subset \mathbb{R}^n$ is forward invariant for system (1) if its solutions for some $\mathbf{u} \in U$ starting at any $\mathbf{x}(0) \in C$ satisfy $\mathbf{x}(t) \in C$, $\forall t \geq 0$.

Definition 3. (Relative degree): The relative degree of a (sufficiently many times) differentiable function $b : \mathbb{R}^n \rightarrow \mathbb{R}$ with respect to system (1) is the number of times it needs to be differentiated along its dynamics until any component of the control $\mathbf{u}$ explicitly shows in the corresponding derivative.

For systems with multiple control inputs, existing CBF methods may fail due to inconsistent relative degrees across control components. In such cases, we may define a relative degree set, which can be addressed by integral CBFs [48] that can make desired (all) control components show up in the derivative. Since function b is used to define a (safety) constraint $b(\mathbf{x}) \geq 0$, we will refer to the relative degree of b as the relative degree of the constraint. For a constraint $b(\mathbf{x}) \geq 0$ with relative degree m , $b : \mathbb{R}^n \rightarrow \mathbb{R}$, and $\psi_0(\mathbf{x}) := b(\mathbf{x})$, we define a sequence of functions $\psi_i : \mathbb{R}^n \rightarrow \mathbb{R}$, $i \in \{1, \dots, m\}$

$$\psi_i(\mathbf{x}) := \dot{\psi}_{i-1}(\mathbf{x}) + \alpha_i(\psi_{i-1}(\mathbf{x})), \quad i \in \{1, \dots, m\} \quad (2)$$

where $\alpha_i(\cdot)$, $i \in \{1, \dots, m\}$ denotes a $(m - i)$ th order differentiable class $\mathcal{K}$ function.

TABLE I
NOTATION

Symbol	Definition
t	Time
$m \in \mathbb{N}$	Relative degree
θ	BarrierNet parameters
$\mathbf{x} \in \mathbb{R}^n$	System state
$\mathbf{z} \in \mathbb{R}^d$	Observation variable
$\mathbf{z}_d \in \mathbb{R}^{(m-1)d}$	Derivatives of $\mathbf{z}$
$\mathbf{u} \in \mathbb{R}^q$	Control
$\alpha : [0, a) \rightarrow [0, \infty)$, $a > 0$	Class $\mathcal{K}$ function
$\beta : \mathbb{R} \rightarrow \mathbb{R}$	Extended class $\mathcal{K}$ function
$b : \mathbb{R}^n \rightarrow \mathbb{R}$	Safety constraint
$\psi : \mathbb{R}^n \rightarrow \mathbb{R}$	CBF
$p : \mathbb{R}^d \rightarrow \mathbb{R}^{>0}$	Penalty function
$C \subset \mathbb{R}^n$	Safe set
$l : \mathbb{R}^q \times \mathbb{R}^q \rightarrow \mathbb{R}$	Similarity measure
$f : \mathbb{R}^n \rightarrow \mathbb{R}^n$	Affine dynamics drift term
$g : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times q}$	Affinedynamics control term

We further define a sequence of sets C_i , $i \in \{1, \dots, m\}$ associated with (2) in the form

$$C_i := \{\mathbf{x} \in \mathbb{R}^n : \psi_{i-1}(\mathbf{x}) \geq 0\}, \quad i \in \{1, \dots, m\}. \quad (3)$$

Definition 4. (HOCBF [12]): Let $C_1, \dots, C_m$ be defined by (3) and $\psi_1(\mathbf{x}), \dots, \psi_m(\mathbf{x})$ be defined by (2). A function $b : \mathbb{R}^n \rightarrow \mathbb{R}$ is an HOCBF of relative degree m for system (1) if there exist $(m - i)$ th order differentiable class $\mathcal{K}$ functions α_i , $i \in \{1, \dots, m - 1\}$ and a class $\mathcal{K}$ function α_m such that

$$\sup_{\mathbf{u} \in U} \left[L_f^m b(\mathbf{x}) + \left[L_g L_f^{m-1} b(\mathbf{x}) \right] \mathbf{u} + O(b(\mathbf{x})) + \alpha_m(\psi_{m-1}(\mathbf{x})) \right] \geq 0 \quad (4)$$

for all $\mathbf{x} \in C_1 \cap \dots \cap C_m$. In (4), L_f^m (L_g) denotes Lie derivatives along f (g) m (one) times, and $O(b(\mathbf{x})) = \sum_{i=1}^{m-1} L_f^i (\alpha_{m-i} \circ \psi_{m-i-1})(\mathbf{x})$. Further, $b(\mathbf{x})$ is such that $L_g L_f^{m-1} b(\mathbf{x}) \neq 0$ on the boundary of the set $C_1 \cap \dots \cap C_m$.

The HOCBF is a general form of the relative degree one CBF [2] (setting $m = 1$ reduces the HOCBF to the common CBF form). We can define $\alpha_i(\cdot)$, $i \in \{1, \dots, m\}$ in Definition 4 to be extended class $\mathcal{K}$ functions to ensure robustness of an HOCBF to perturbations [2].

Theorem 1. ([12]): Given an HOCBF $b(\mathbf{x})$ from Definition 4 with the associated sets $C_1, \dots, C_m$ defined by (3), if $\mathbf{x}(0) \in C_1 \cap \dots \cap C_m$, then any Lipschitz continuous controller $\mathbf{u}(t)$ that satisfies the constraint in (4), $\forall t \geq 0$ renders $C_1 \cap \dots \cap C_m$ forward invariant for system (1).

We provide a summary of notations in Table I.

IV. PROBLEM FORMULATION

We make the following assumptions in this work:

Assumption 1:

- 1) All measurements are assumed to be precise and reliable, and there are no occluded objects.
- 2) The environment model is assumed to be accessible without time delay.
- 3) The system is not subject to any disturbances.
- 4) The system dynamics (1) are assumed to accurately match the real system.

- 5) The robot moves in low speed such that **the control bounds are independent from each other.**

The above assumptions can be addressed using the online verification method [22], but this approach tends to be computationally expensive. These assumptions can also be addressed using the efficient event-triggered control framework [29], in which the issues of unknown dynamics, disturbances, and measurement uncertainties are considered. This event-triggered framework can be directly applied to the proposed BarrierNet, and thus, we only focus on the theoretic foundations in this work. For reliability of perception and occluded objects, we may consider multisensors or infer from other observed participants, and this will be further studied in future work.

Here, we formally define the learning problem for safety-critical control.

Problem 1: Given:

- 1) a system with known affine dynamics in the form of (1);
- 2) a state-feedback nominal controller $h^*(x) = u^*$ (such as the model predictive controller) that is taken as the training label;
- 3) **a set of safety constraints $b_j(x) \geq 0, j \in S$ (where b_j is continuously differentiable, S is a constraint set);**
- 4) control bounds $u_{\min} \leq u \leq u_{\max}$;
- 5) a neural network controller $h(x|\theta) = u$ parameterized by θ ;

our goal is to find the optimal parameters

$$\theta^* = \arg \min_{\theta} \mathbb{E}_x[l(h^*(x), h(x|\theta))]$$

while guaranteeing the satisfaction of the safety constraints in 3) and control bounds in 4). $\mathbb{E}(\cdot)$ is the expectation and $l(\cdot, \cdot)$ denotes a similarity measure.

Problem 1 defines policy distillation with safety guarantees. The safety constraints can be predefined by users or they can be learned [33], [34].

V. BARRIERNET

We introduce BarrierNet—a CBF-based neural network controller with parameters trainable via backpropagation. We define the safety guarantees of a neural network controller as follows.

Definition 5: (Safety guarantees) A neural network controller has safety guarantees for system (1) if its outputs (controls) satisfy control bounds 4) in Problem 1 and drive system (1) such that $b_j(x(t)) \geq 0, \forall t \geq 0, \forall j \in S$.

BarrierNet addresses several limitations with existing methods. The HOCBF method provides safety guarantees for control systems with arbitrary relative degrees but in a conservative way. In other words, the satisfaction of the HOCBF constraint (4) is only a sufficient condition for the satisfaction of the original safety constraint $b(x) \geq 0$. This conservativeness of the HOCBF method significantly limits the system's performance. For example, conservativeness may drive the system much further away from obstacles than necessary. Our first motivation for BarrierNet is to address this conservativeness.

More specifically, an HOCBF constraint is always a hard constraint in order to guarantee safety. This may adversely affect

the performance of the system. For instance, if a class $\mathcal{K}$ function α_i in the HOCBF is too steep (like a step function), then they are the least conservative. However, such a HOCBF constraint (4) becomes active only near the unsafe set boundary, requiring a large control input effort. Hence, there might not exist a feasible control that satisfies the HOCBF constraint (4) and the control bound at the same time. If, on the other hand, α_i is too flat, then the HOCBF is over-conservative as the HOCBF constraint becomes active when the system is far from the unsafe set boundary. Ideally, we wish to *find a α_i that has the steepest slope while making the HOCBF constraint compatible with the control bound when it becomes active.* In such cases, we call a HOCBF *nonconservative*. **The current approaches [30], [31] only consider the feasibility issue of the CBF method without considering this conservativeness.**

BarrierNet learns an HOCBF without losing the safety guarantees. In addition, we also incorporate HOCBF in a differentiable optimization layer to allow the tuning of its parameters from data. Given a safety requirement $b(x) \geq 0$ with relative degree m for system (1), we redefine the sequence of CBFs in (2) as

$$\psi_i(x, z, z_d) := \psi_{i-1}(x, z, z_d) + p_i(z)\alpha_i(\psi_{i-1}(x, z, z_d)) \quad i \in \{1, \dots, m\} \quad (5)$$

where $\psi_0(x, z) = b(x)$. The variable $z \in \mathbb{R}^d$ **is the input of the neural network** ($d \in \mathbb{N}$ is the dimension of the features), and it is assumed that the relative degree of each component in z is not lower than that of the safety constraint (this makes sure that **no control will appear in the derivatives of z** during construction of an HOCBF), which is reasonable since z often comes from high-dimensional sensory measurement like images; otherwise, its derivative can be omitted, as shown later. $z_d = (z^{(1)}, \dots, z^{(m-1)}) \in \mathbb{R}^{(m-1)d}$ denotes the derivatives of the input z ; $p_i: \mathbb{R}^d \rightarrow \mathbb{R}^{>0}, i \in \{1, \dots, m\}$ are the outputs of the previous layer (e.g., CNN, LSTM, or MLP, as shown in Fig. 12) or trainable parameters themselves (i.e., p_i is independent of z), where $\mathbb{R}^{>0}$ denotes the set of positive scalars. Note that it is possible to absorb $p_i(z)$ into the class $\mathcal{K}$ function α_i for notational simplicity, but it may raise the question of how to learn a class $\mathcal{K}$ function. We adopt the most general and flexible formulation that allows arbitrary class $\mathcal{K}$ function definition. If $p_i, i \in \{1, \dots, m\}$ are just trainable parameters, then the above learnable CBFs are not involved with z_d . The above formulation is similar to AdaCBFs [32] and can still guarantee safety, but is trainable and does not require us to design auxiliary dynamics for p_i (that an AdaCBF does require), which is a challenging aspect of the existing AdaCBF method. Then, we have a similar HOCBF constraint (called differentiable CBF (dCBF)) as in Definition 4 of the form

$$L_f^m b(x) + \left[L_g L_f^{m-1} b(x) \right] u + O(b(x), z, z_d) + p_m(z)\alpha_m(\psi_{m-1}(x, z, z_d)) \geq 0 \quad (6)$$

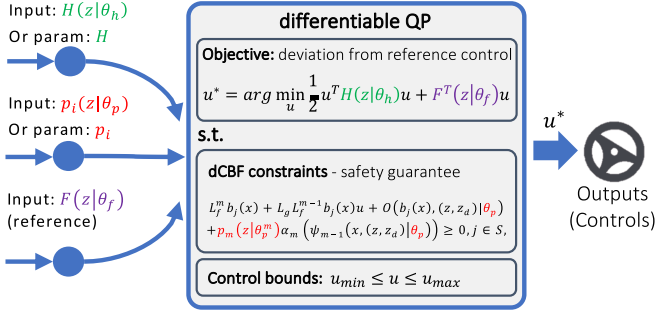


Fig. 3. BarrierNet structure. F serves as a reference output (control), and all the other parameters could be either trainable (parameter of the BarrierNet) or depend on the previous layers (input of the BarrierNet). Note that we can compute reference control as $H^{-1}F$ using reference F .

Given learnable dCBFs defined by (5) and (6) that are not conservative, we can incorporate them as a differentiable optimization layer, which is our BarrierNet.

Definition 6 (BarrierNet): A BarrierNet is composed by neurons in the form

$$\mathbf{u}^*(t) = \arg \min_{\mathbf{u}(t)} \frac{1}{2} \mathbf{u}(t)^T \mathbf{H}(\mathbf{z}|\theta_h) \mathbf{u}(t) + \mathbf{F}^T(\mathbf{z}|\theta_f) \mathbf{u}(t) \quad (7)$$

s.t.

$$\begin{aligned} & L_f^m b_j(\mathbf{x}) + [L_g L_f^{m-1} b_j(\mathbf{x})] \mathbf{u} + O(b_j(\mathbf{x}), (\mathbf{z}, \mathbf{z}_d)|\theta_p) \\ & + p_m(\mathbf{z}|\theta_p^m) \alpha_m(\psi_{m-1}(\mathbf{x}, (\mathbf{z}, \mathbf{z}_d)|\theta_p)) \geq 0, j \in S \\ & \mathbf{u}_{\min} \leq \mathbf{u} \leq \mathbf{u}_{\max}, \quad t = k\Delta t + t_0 \end{aligned} \quad (8)$$

where $H(\mathbf{z}|\theta_h) \in \mathbb{R}^{q \times q}$ is positive definite, $H^{-1}(\mathbf{z}|\theta_h)F(\mathbf{z}|\theta_f) \in \mathbb{R}^q$ could be interpreted as a reference control (the output of previous network layers) and $\theta_h, \theta_f, \theta_p = (\theta_p^1, \dots, \theta_p^m)$ are trainable parameters, $\Delta t > 0$ is the discretization time.

At a high level, this formulation produces control close to that of the nominal controller while satisfying dCBF constraints, guaranteeing safety with minimal deterioration in performance. The inequality (8) in Definition 6 guarantees each safety constraint $b_j(\mathbf{x}) \geq 0, \forall j \in S$ through the parameterized function $p_i, i \in \{1, \dots, m\}$. Based on Definition 6, for instance, if we have ten control agents, we need ten BarrierNet neurons presented by (7) to ensure the safety of each agent. This implies that BarrierNet can be extended to multiagent settings.

In Definition 6, we make $H(\mathbf{z}|\theta_h)$ parameterized and dependent on the network input $\mathbf{z}$, but H can also be directly trainable parameters that do not depend on the previous layer (i.e., we have H). The same applies to $p_i, i \in \{1, \dots, m\}$. The trainable parameters are $\theta = \{\theta_h, \theta_f, \theta_p\}$ (or $\theta = \{H, \theta_f, p_i, \forall i \in \{1, \dots, m\}\}$ if H and p_i do not depend on the previous layer). The solution $\mathbf{u}^*$ is the output of the neuron. The whole structure of a BarrierNet is shown in Fig. 3. The BarrierNet is differentiable with respect to its parameters [38]. We describe the forward and backward passes as follows.

A. Forward Pass

The forward step of a BarrierNet is to solve the QP in Definition 6. The inputs of a BarrierNet include environmental features $\mathbf{z}$ (such as the location and speed of an obstacle) that can be provided directly or from a tracking network if raw sensory inputs are used. BarrierNet also takes as inputs the system states $\mathbf{x}$ as feedback, as shown in Fig. 1. The outputs are the solutions of the QP (the resultant controls).

B. Backward Pass

The main task of BarrierNet is to provide controls while always ensuring safety. Suppose ℓ denotes some loss function (a similarity measure). Using the Lagrangian of QP followed by applying the Karush–Kuhn–Tucker conditions, as introduced in [38], we can find the loss gradient with respect to all the parameters. Specifically, let λ denote the dual variables on the HOCBF constraints, and $D(\cdot)$ creates a diagonal matrix from a vector, and let $\mathbf{u}^*, \lambda^*$ denote the optimal solutions of $\mathbf{u}$ and λ , respectively, we can first get $d_{\mathbf{u}}$ and d_{λ} in the form

$$\begin{bmatrix} d_{\mathbf{u}} \\ d_{\lambda} \end{bmatrix} = \begin{bmatrix} H & G^T D(\lambda^*) \\ G & D(G\mathbf{u}^* - h) \end{bmatrix}^{-1} \begin{bmatrix} (\frac{\partial \ell}{\partial \mathbf{u}^*})^T \\ 0 \end{bmatrix} \quad (9)$$

where G, h are concatenated by $G_j, h_j, j \in S$, where

$$\begin{aligned} G_j &= -L_g L_f^{m-1} b_j(\mathbf{x}) \\ h_j &= L_f^m b_j(\mathbf{x}) + O(b_j(\mathbf{x}), \mathbf{z}, \mathbf{z}_d) \\ &+ p_m(\mathbf{z}) \alpha_m(\psi_{m-1}(\mathbf{x}, \mathbf{z}, \mathbf{z}_d)). \end{aligned} \quad (10)$$

Since the control bounds in (7) are not trainable, they are not included in G, h .

Then, the relevant gradient with respect to all the BarrierNet parameters can be given by¹

$$\begin{aligned} \nabla_H \ell &= \frac{1}{2} (d_{\mathbf{u}} \mathbf{u}^T + \mathbf{u} d_{\mathbf{u}}^T), \quad \nabla_F \ell = d_{\mathbf{u}} \\ \nabla_G \ell &= D(\lambda^*) (d_{\lambda} \mathbf{u}^T + \lambda d_{\mathbf{u}}^T), \quad \nabla_h \ell = -D(\lambda^*) d_{\lambda}. \end{aligned} \quad (11)$$

In the above equation, $\nabla_G \ell$ is not applicable in a BarrierNet as it is determined by the corresponding HOCBF. $\nabla_h \ell$ is also not directly related to the input of a BarrierNet. Nevertheless, we have $\nabla_{p_i} \ell = \nabla_{h_j} \ell \nabla_{p_i} h_j, i \in \{1, \dots, m\}, j \in S$, where $\nabla_{h_j} \ell$ is given by $\nabla_h \ell$ in (11) and $\nabla_{p_i} h_j$ is given by taking the partial derivative of h_j in (10).

The following theorem characterizes the safety guarantees of a BarrierNet.

Theorem 2: If $p_i(\mathbf{z}), i \in \{1, \dots, m\}$ are differentiable (further, the relative degree of each component in $\mathbf{z}$ is not lower than that of the safety constraint) or $p_i, i \in \{1, \dots, m\}$ are parameters to be trained, then a BarrierNet composed by neurons as in Definition 6 guarantees the safety of system (1).

Proof: If $p_i, i \in \{1, \dots, m\}$ are trainable parameters (i.e., p_i is independent of $\mathbf{z}$), then the dCBF (6) is just a regular HOCBF whose parameters are optimally determined by the training data, and thus, safety is guaranteed for system (1) by Theorem 1.

¹Note: the gradient with respect to the parameters θ_h, θ_f , and θ_p can be obtained using the chain rule.

If $p_i(z), i \in \{1, \dots, m\}$ are differentiable and the relative degree of each component in z is not lower than that of the safety constraint (no control will appear in the derivatives of z during construction of an HOCBF), it follows from Theorem 1 that each $\psi_i(x, z, z_d)$ in (5) is a valid CBF. Starting from $\psi_m(x, z, z_d) \geq 0$ [the nonLie-derivative form of each HOCBF constraint (6)], we can show that $\psi_{m-1}(x, z, z_d) \geq 0$ is guaranteed to be satisfied following Theorem 1. Recursively, we can show that $\psi_0(x, z) \geq 0$ is guaranteed to be satisfied. As $b(x) = \psi_0(x, z)$ following (5), we have that system (1) is safety guaranteed in a BarrierNet. ■

Consider the case that $p_i(z), i \in \{1, \dots, m\}$ depend on the input. In order to guarantee that $p_i(z), i \in \{1, \dots, m\}$ are continuously differentiable and positive, we can choose differentiable activation functions for the previous layers, such as *sigmoid functions*. The derivatives of the penalty functions $p_i(z), i \in \{1, \dots, m\}$, i.e., the derivatives of z may be hard to evaluate. For instance, if z contains the pixels of an image, then its derivative is inaccurate when the camera sampling frequency is low. Nevertheless, we have the following corollary to show the safety guarantees of the BarrierNet.

Corollary 1: If $z_d = 0$, the class $\mathcal{K}$ functions $\alpha_i(\cdot), i \in \{1, \dots, m\}$ in (5) are linear functions, and $p_i(z), i \in \{1, \dots, m\}$ are such that $x \in C_1 \cap \dots \cap C_m$, then a BarrierNet composed by neurons as in Definition 6 guarantees the safety of system (1).

Proof: When the class $\mathcal{K}$ functions $\alpha_i(\cdot), i \in \{1, \dots, m\}$ in (5) are linear functions, the dCBF constraint (8) can be rewritten as

$$L_f^m b_j(x) + [L_g L_f^{m-1} b_j(x)]u + \sum_{r=1}^m \left[\sum_{1 \leq k_1 < k_2 < \dots < k_r \leq m} \left(\prod_{l=1}^r p_{k_l}(z) \right) + P_r(p) \right] b_j^{(m-r)}(x) \geq 0 \quad (12)$$

where $b_j^{(0)}(x) = b_j(x)$ and $P_r(p), r \in \{1, \dots, m\}$ are the polynomials of the derivatives of $p_i(z), i \in \{1, \dots, m\}$, such that $z_d = 0$ implies $P_r(p) = 0, \forall r \in \{1, \dots, m\}$. Note that when setting $z_d = 0$, it is equivalent to setting p_i to be piecewise constant among all discretized time intervals, and the value of p_i within each time interval is determined by the value of the observation z at the beginning of the discretized time interval. If z is dependent on x , in order to make the control u not show up in z_d [equivalently, $P_r(p)$], we may use the observation z such that $p_i(z)$ has a relative degree that is not lower than the one of the safety constraint.

According to the exponential CBF [3], a system is safety guaranteed in terms of the constraint $b_j(x) \geq 0$ if the polynomial

$$s^m + \sum_{r=0}^{m-1} l_r s^r = 0 \quad (13)$$

corresponding to

$$L_f^m b_j(x) + [L_g L_f^{m-1} b_j(x)]u + \sum_{r=0}^{m-1} l_r b_j^{(r)}(x) \geq 0 \quad (14)$$

has only negative real roots and $p_i(z), i \in \{1, \dots, m\}$ are such that $x \in C_1 \cap \dots \cap C_m$.

As shown in (12), if $z_d = 0$, then $P_r(p) = 0$, i.e., (12) can be rewritten as

$$L_f^m b_j(x) + [L_g L_f^{m-1} b_j(x)]u + \sum_{r=1}^m \left[\sum_{1 \leq k_1 < k_2 < \dots < k_r \leq m} \left(\prod_{l=1}^r p_{k_l}(z) \right) \right] b_j^{(m-r)}(x) \geq 0. \quad (15)$$

Combining (15) and (14), and by Vieta's formulas, we have that $-p_i(z), i \in \{1, \dots, m\}$ ($p_i(z) > 0$) are the negative real roots for the polynomial (13). Therefore, the condition (15) implies the satisfaction of $b_j(x) \geq 0$, and the BarrierNet composed by neurons as in Definition 6 guarantees the safety of system (1). ■

In order to get piece-wise constant penalties $p_i(z)$ (i.e., $z_d = 0$) in dCBFs, we can determine the value of $p_i(z)$ and make it a constant before the construction of dCBFs, which can avoid taking the derivatives of $p_i(z)$. If we do wish to consider z_d in the BarrierNet, then the uncertainty of z_d can be addressed by considering the inter-sampling effect (i.e., constraint satisfaction between discrete time instant) [28], [29] when the bounds of z_d are known. We have shown the safety guarantees of a BarrierNet with linear class $\mathcal{K}$ functions when $z_d = 0$ by Corollary 1. We can also infer that safety is still guaranteed for other types of class $\mathcal{K}$ functions in a BarrierNet. $p_i(z)$ can be truncated to satisfy $x \in C_1 \cap \dots \cap C_m$, and this is discussed in [12]. If no such $p_i(z)$ exists (such as due to uncertainties), then we can define all class $\mathcal{K}$ functions to be the extended class $\mathcal{K}$ functions [such as the linear functions shown in (15)] to achieve robust control [2].

Remark 1 (Adaptivity of the BarrierNet): The HOCBF constraints in a BarrierNet are regulated by the trainable penalty functions without losing safety guarantees. The penalty functions are environment-dependent. Their features can be calculated from upstream networks. The adaptive property of the HOCBFs provides the adaptiveness of the BarrierNet. As a result, BarrierNet is able to generate safe controls while avoiding overly conservative behavior.

Remark 2 (Feasibility guarantees of a BarrierNet): Due to the existence of the control bound in a BarrierNet, the dCBF-based QP (7) may become infeasible because of the possible conflict between the control bound and the dCBF constraints. In order to address this, we may require that the nominal controller provide ground truth (i.e., control labels) that strictly satisfies the safety constraints and the control bounds. Then, during the training of the BarrierNet layer, we can relax/remove the control bounds. After the neural network converges, the differentiable QPs are feasible when we add control bounds in the testing or implementation. However, there is still the possibility that the QP could be infeasible as the BarrierNet may have some inputs that it had not seen before. However, we can find sufficient conditions of feasibility, as shown in [31]. Briefly, this approach finds a feasibility constraint on the state of the system along with the penalties $p_i(z), i \in \{1, \dots, m\}$, and then enforces this

Algorithm 1: Construction and Training of BarrierNet.

Input: Dynamics (1), Safety requirements in Problem 1, a nominal controller

Output: A BarrierNet with optimal parameters θ^* .

(a) Construct learnable HOCBFs by (5), and construct the BarrierNet by (7)

(b) Get the training data set using the nominal controller

(c) Initialize the BarrierNet parameter θ , the Epochs, and the learning rate γ

while e in Epochs **do**

 Forward: Solve (7) and get the loss ℓ

 Backward: Get the loss gradient $\nabla_{\theta}\ell$ using (11)

$\theta \leftarrow \theta - \gamma \nabla_{\theta}\ell$

end

return A BarrierNet with optimal parameters θ^*

feasibility constraint using another CBF. For instance, one of the sufficient conditions for the feasibility guarantee of the adaptive cruise control problem shown in [31] is $v \leq v_p + \frac{c_{dg}(p_1+p_2)}{p_1 p_2}$, where v, v_p denote the velocities of the ego and the preceding vehicles, respectively, $c_{dg} > 0$ denotes the maximum braking of the ego vehicle. However, it is still possible that no feasible solutions can be found under complex specifications. In such cases, we assign priorities to specifications and make sure the most important specifications (such as no collision with pedestrians in driving) are always satisfied while relaxing the less important specifications (such as lane keeping). The relaxation of less important specifications will not affect how we obtain CBFs as this is done by relaxing the corresponding CBF constraint (i.e., replacing the right hand 0 of (8) by a slack variable, and minimizing the slack variable in the cost as well). The CBFs can be obtained using the disk-covering approach [49], which has been shown to work for all kinds of traffic participants. The CBF method has been shown to work in complex specifications [49], and thus, applicable to the BarrierNet using the priority structure.

Remark 3: The complexity of the BarrierNet is $\mathcal{O}(d^3)$ using interior point method [50], where d is the dimension of the decision variable (control input) in the QP. Empirically, the number of constraints does not significantly affect the complexity. Moreover, the batch-QP solving method introduced in [38] makes the BarrierNet tractable to be trained on large datasets using batch training method. Therefore, the proposed BarrierNet is scalable to more inputs and constraints and can satisfy requirements for actual deployments.

The process of constructing and training a BarrierNet includes the following.

- 1) Construct a learnable HOCBF by (5) that enforces each of the safety requirements.
- 2) Construct the parameterized BarrierNet by (7).
- 3) Get the training dataset using the nominal controller.
- 4) Train the BarrierNet using error backpropagation.

We summarize the algorithm for the BarrierNet in Algorithm 1.

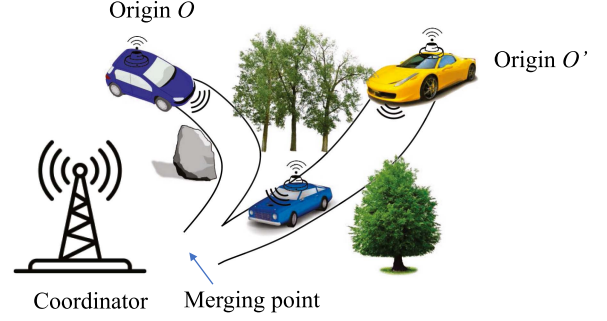


Fig. 4. Traffic merging problem. A collision may happen at the MP as well as everywhere within the CZ.

VI. EXPERIMENTS AND DISCUSSIONS

In this section, we present three case studies (a traffic merging control problem and robot navigation problems in 2-D and 3-D) to verify the effectiveness of our proposed BarrierNet.

A. Traffic Merging Control

1) *Experiment Setup:* The traffic merging problem arises when traffic must be joined from two different roads, usually associated with the main lane and a merging lane, as shown in Fig. 1. We consider the case where all traffic consisting of controlled autonomous vehicles (CAVs) arrive randomly at the origin (O and O') and join at the merging point (MP) M where a lateral collision may occur. The segment from the origin to the MP M has length L for both lanes and is called the control zone (CZ). All CAVs do not overtake each other in the CZ as each road consists of a single lane. A coordinator is associated with the MP, whose function is to maintain a first-in-first-out (FIFO) queue of CAVs based on their arrival time at the CZ. The coordinator also enables real-time communication among the CAVs that are in the CZ, including the last one leaving the CZ. The FIFO assumption imposed so that CAVs cross the MP in their order of arrival is made for simplicity and often to ensure fairness.

2) *Notation:* x_k, v_k , and u_k denote the along-lane position, speed, and acceleration (control) of CAV k , respectively. t_k^0 and t_k^m denote the arrival time of CAV k at the origin and the MP, respectively. z_{k,k_p} denotes the along lane distance between CAV k and its preceding CAV k_p , as shown in Fig. 4.

Our goal is to jointly minimize all vehicles' travel time and energy consumption in the CZ. Written as an objective function, we have

$$\min_{u_k(t)} \beta(t_k^m - t_k^0) + \int_{t_k^0}^{t_k^m} \frac{1}{2} u_k^2(t) dt \quad (16)$$

where u_k is the vehicle's control (acceleration), and $\beta > 0$ is a weight controlling the relative magnitude of travel time and energy consumption. We assume double integrator dynamics for all vehicles.

Each vehicle k should satisfy the following rear-end safety constraint if its preceding vehicle k_p is in the same lane

$$z_{k,k_p}(t) \geq \phi v_k(t) + \delta, \quad \forall t \in [t_k^0, t_k^m] \quad (17)$$

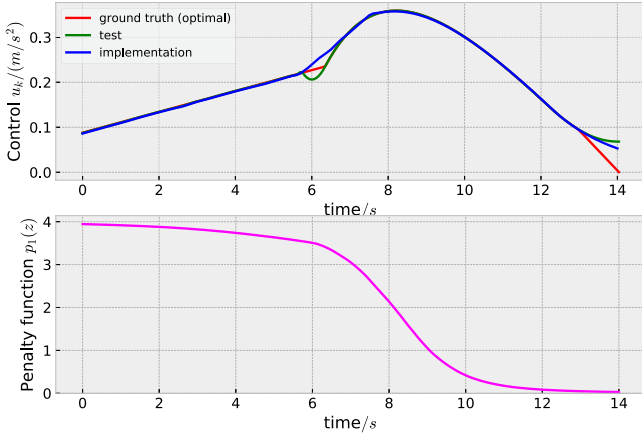


Fig. 5. Control and penalty function $p_1(z)$ from the BarrierNet when training with the optimal controller. The blue curves (labeled as implementation) are the vehicle control when we apply the BarrierNet to drive the vehicle dynamics to pass through the CZ.

where $z_{k,k_p} = x_{k_p} - x_k$ denotes the along-lane distance between k and k_p , ϕ is the reaction time (usually takes 1.8 s) and $\delta \geq 0$.

The traffic merging problem is to find an optimal control that minimizes (16), subject to (17). We assume vehicle k has access to only the information of its immediate neighbors from the coordinator (shown in Fig. 4), such as the preceding vehicle k_p . This merging problem can be solved analytically by optimal control methods [51], but at the cost of extensive computation, and the solution becomes complicated when one or more constraints become active in an optimal trajectory, hence possibly prohibitive for real-time implementation. In the worst case, the safety constraint may recursively become active (such as in heavy traffic), which significantly increases the computation time.

3) *BarrierNet Design*: We enforce the safety constraint (17) by a CBF $b(z_{k,k_p}, v_k) = z_{k,k_p}(t) - \phi v_k(t) - \delta$, and any control input u_k should satisfy the CBF constraint (4) which in this case (choose α_1 as a linear function in Definition 4) is

$$\varphi u_k(t) \leq v_k(t) - v_{k_p}(t) + p_1(z)(z_{k,k_p}(t) - \phi v_k(t) - \delta) \quad (18)$$

where v_k is the speed of vehicle k and $z = (x_{k_p}, v_{k_p}, x_k, v_k)$ is the input of the neural network model (to be designed later). $p_1(z)$ is called a penalty in the CBF that addresses the conservativeness of the CBF method. The cost in the neuron of the BarrierNet is given by

$$\min_{u_k} (u_k - f_1(z))^2 \quad (19)$$

where $f_1(z)$ is a reference to be trained [the output of the fully connected (FC) network]. Then, we create a neural network model whose structure is composed by an FC network (an input layer and two hidden layers) followed by a BarrierNet. The input of the FC network is z , and its output is the penalty $p_1(z)$ and the reference $f_1(z)$. While the input of the BarrierNet is the penalty $p_1(z)$ and the reference $f_1(z)$, and its output is applied to control a vehicle k in the CZ.

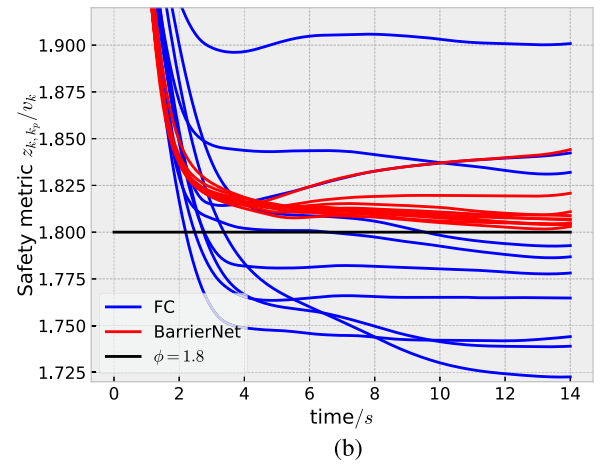
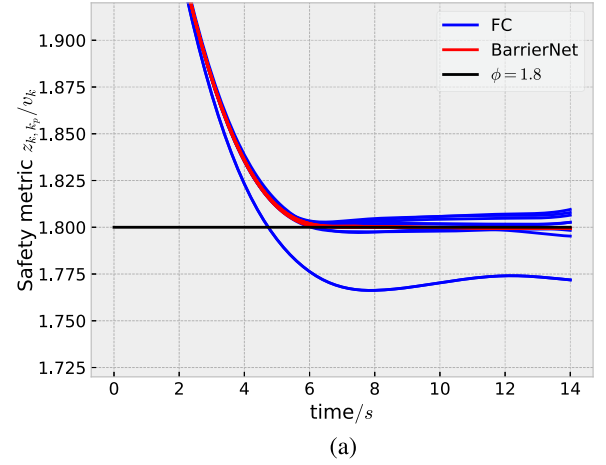


Fig. 6. Safety comparison (under ten trained models) between the BarrierNet and an FC network when training using the optimal/OCBF controller ($\delta = 0$). If $z_{k,k_p}/v_k$ is above the line $\phi = 1.8$, then safety is guaranteed. We observe that only neural network agents equipped with BarrierNet satisfy this condition. (a) Training using the optimal controller. (b) Training using the OCBF controller.

4) *Results and Discussion*: To get the training dataset, we solve an optimal or joint optimal control and barrier function (OCBF) [52] controller offline. The solutions of an optimal or OCBF controller are taken as labels. The training results with the optimal controller and the OCBF controller are shown in Figs. 5–7.

In an optimal controller, the original safety constraint is active after around 6 s, as shown in Fig. 5. Therefore, the sampling trajectory is on the safety boundary, and the inter-sampling effect becomes important in this case. Since we do not consider the inter-sampling effect in this article, the safety metric of the BarrierNet might go below the lower bound $\phi = 1.8$, as the red curves shown in Fig. 6(a). However, due to the Lyapunov property of the CBF, the safety metric will always stay close to the lower bound $\phi = 1.8$. The solutions for ten trained models are also consistent. In an FC network, the safety metrics vary under different trained models, and the safety constraint might be violated, as the blue curves are shown in Fig. 6(a).

In an OCBF controller, the original safety constraint is not active, and thus, the inter-sampling effect is not sensitive. As

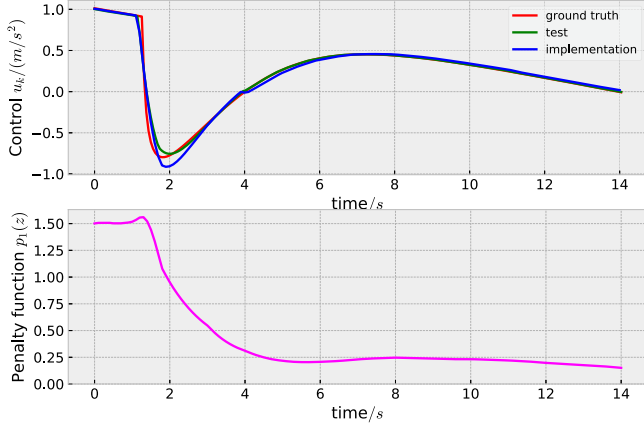


Fig. 7. Control and penalty function $p_1(z)$ from the BarrierNet when training with the OCBF controller. The blue curves (labeled as implementation) are the vehicle control when we apply the BarrierNet to drive the vehicle dynamics to pass through the CZ.

TABLE II
COMPARISONS BETWEEN BARRIERNET, FC, OPTIMAL CONTROLLER, AND OCBF CONTROLLER

Irem	Compute time	Safety guarantee	Optimality	Adaptive
BarrierNet	< 0.01s	Yes	Close-optimal	Yes
FC	< 0.01s	No	Close-optimal	No
Optimal	1s – 30s	Yes	Optimal	Yes
OCBF	< 0.01s	Yes	Suboptimal	No

shown in Fig. 6(b), safety is always guaranteed in a BarrierNet under ten trained models. While in an FC network, the safety constraint may be violated as there are no guarantees.

We present the penalty functions when training with the optimal controller and the OCBF controller in Figs. 5 and 7, respectively. The penalty function $p_1(z)$ decreases when the CBF constraint becomes active. This shows the adaptivity of the BarrierNet. This behavior is similar to the AdaCBF, but in the BarrierNet, we do not need to design auxiliary dynamics for the penalty functions. Therefore, the BarrierNet is simpler than the AdaCBF. Finally, we present a comprehensive comparison between the BarrierNet, the FC network, the optimal controller, and the OCBF controller in Table II.

B. Two-Dimensional Robot Navigation

1) *Experiment Setup*: We consider a robot navigation problem with obstacle avoidance. In this case, we consider nonlinear dynamics with two control inputs and nonlinear safety constraints. The robot navigates according to the following unicycle model for a wheeled mobile robot:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{v} \end{bmatrix} = \begin{bmatrix} v \cos(\theta) \\ v \sin(\theta) \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \quad (20)$$

where $\mathbf{x} := (x, y, \theta, v)$, $\mathbf{u} = (u_1, u_2)$, x, y denote the robot's 2-D coordinates, θ denotes the heading angle of the robot, v denotes the linear speed, and u_1, u_2 denote the two control inputs for turning and acceleration.

2) *BarrierNet Design*: The robot is required to avoid a circular obstacle in its path, i.e., the state of the robot should satisfy

$$(x - x_o)^2 + (y - y_o)^2 \geq R^2 \quad (21)$$

where $(x_o, y_o) \in \mathbb{R}^2$ denotes the location of the obstacle, and $R > 0$ is the radius of the obstacle.

The goal is to minimize the control input effort while subject to the safety constraint (21) as the robot approaches its destination.

The relative degree of the safety constraint (21) is 2 with respect to the dynamics (20), thus, we use an HOCBF $b(\mathbf{x}) = (x - x_o)^2 + (y - y_o)^2 - R^2$ to enforce it. Any control input $\mathbf{u}$ should satisfy the HOCBF constraint (4) which in this case (choose α_1, α_2 in Definition 4 as linear functions) is

$$\begin{aligned} -L_g L_f b(\mathbf{x}) \mathbf{u} &\leq L_f^2 b(\mathbf{x}) + (p_1(z) + p_2(z)) L_f b(\mathbf{x}) \\ &+ (\dot{p}_1(z) + p_1(z) p_2(z)) b(\mathbf{x}) \end{aligned} \quad (22)$$

where

$$\begin{aligned} L_g L_f b(\mathbf{x}) &= [-2(x - x_o)v \sin \theta + 2(y - y_o)v \cos \theta \\ &\quad 2(x - x_o) \cos \theta + 2(y - y_o) \sin \theta] \\ L_f^2 b(\mathbf{x}) &= 2v^2 \\ L_f b(\mathbf{x}) &= 2(x - x_o)v \cos \theta + 2(y - y_o)v \sin \theta. \end{aligned} \quad (23)$$

In the above equations, $\mathbf{z} = (\mathbf{x}, x_d)$ is the input to the model, where $x_d \in \mathbb{R}^2$ is the location of the destination, and $p_1(z), p_2(z)$ are the trainable penalty functions. $\dot{p}_1(\mathbf{x})$ is set as 0.

The cost in the neuron of the BarrierNet is given by

$$\min_{\mathbf{u}} (u_1 - f_1(z))^2 + (u_2 - f_2(z))^2 \quad (24)$$

where $f_1(z)$ and $f_2(z)$ are reference controls provided by the upstream network (the outputs of the FC network).

3) *Results and Discussion*: The training data are obtained by solving the CBF controller introduced in [12], and we generate 100 trajectories of different destinations as the training dataset. We compare the FC model, the deep forward-backward model (DFB) [37] that is equivalent to taking the CBF-based QP as a safety filter, and our proposed BarrierNet. The training and testing results are shown in Fig. 8(a)–(d). All the models are trained for obstacle size $R = 6$ m. The controls from the BarrierNet can stay very close to the ground truth, while there are jumps for controls from the DFB when the robot gets close to the obstacle, which shows the conservativeness of the DFB, as shown by the blue solid (BarrierNet) and blue dashed (DFB) curves in Fig. 8(a) and (b). The robot trajectory (dashed blue) from the DFB stays far away from ground truth in Fig. 8(d), and this again shows its conservativeness. The robot from the FC will collide with the obstacle as there is no safety guarantee, as the dotted-blue line shown in Fig. 8(d).

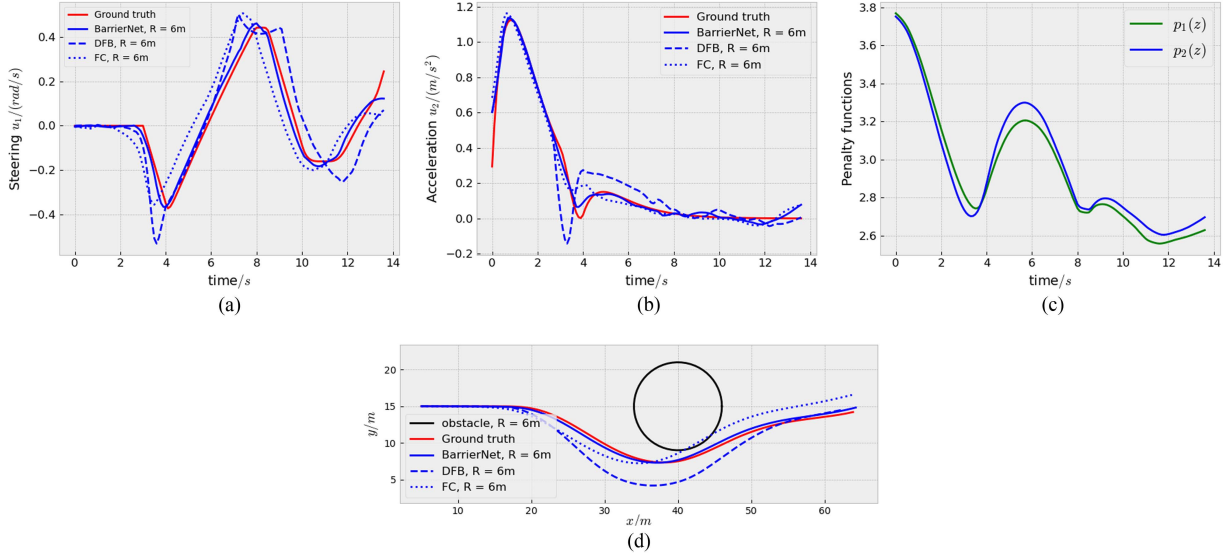


Fig. 8. Controls and trajectories from the FC, DFB, and BarrierNet under the training obstacle size $R = 6$ m. The results refer to the case that the trained FC/DFB/BarrierNet controller is used to drive a robot to its destination. Safety is guaranteed in both DFB and BarrierNet models but not in the FC model. The DFB tends to be more conservative such that the trajectories/controls stay away from the ground truth as its CBF parameters are not adaptive. The varying penalty functions allow the generation of desired control signals and trajectories (given by training labels) and demonstrate the adaptivity of the BarrierNet with safety guarantees. (a) Control u_1 . (b) Control u_2 . (c) Penalty functions. (d) Robot trajectories.

When we increase the obstacle size during implementation (i.e., the trained BarrierNet/DFB/FC controller is used to drive a robot to its destination), the controls u_1, u_2 from the BarrierNet and DFB deviate from the ground truth, as shown in Fig. 9(a) and (b). This is due to the fact that the BarrierNet and DFB will always ensure safety first. Therefore, safety is always guaranteed in the BarrierNet and DFB, as the solid and dashed curves shown in Fig. 10(a). Both the BarrierNet and DFB show some adaptivity to the size change of the obstacle, while the FC controller cannot be adaptive to the size change of the obstacle. Thus, the safety constraint (21) will be violated, as shown by the dotted curves in Fig. 10(a).

The difference between the DFB and the proposed BarrierNet is in the performance. In Fig. 10(b), we show all the trajectories from the BarrierNet, DFB, and FC controllers under different obstacle sizes. Collisions are avoided under the BarrierNet and DFB controllers, as shown by all the solid and dashed trajectories and the corresponding obstacle boundaries in Fig. 10(b). However, as shown in Fig. 10(b), the trajectories from the BarrierNet (solid) can stay closer to the ground truth (red-solid) than the ones from the DFB (dashed) when $R = 6$ m (and other R values). This is due to the fact that the CBFs in the DFB may not be properly defined such that the CBF constraint is active too early when the robot gets close to the obstacle. It is important to note that the robot does not have to stay close to the obstacle boundary under the BarrierNet controller, and this totally depends on the ground truth. The definitions of CBFs in the proposed BarrierNet depend on the environment (network input), and thus, they are adaptive and are without conservativeness.

The profiles of the penalty functions $p_1(z)$ and $p_2(z)$ in the BarrierNet are shown in Fig. 8(c). The values of the penalty functions vary when the robot approaches the obstacle and gets to its destination, and it shows the adaptivity of the BarrierNet

in the sense that with the varying penalty functions, a BarrierNet can produce desired control signals given by labels (ground truth). This is due to the fact the varying penalty functions change the slop of class $\mathcal{K}$ functions in the HOCBF constraint without losing safety guarantees.

C. Three-Dimensional Robot Navigation

1) *Experiment Setup*: We consider a robot navigation problem with obstacle avoidance in 3-D space. In this case, we consider complicated superquadratic safety constraints. The robot navigates according to the double-integrator dynamics. The state of the robot is $\mathbf{x} = (p_x, v_x, p_y, v_y, p_z, v_z) \in \mathbb{R}^6$, in which the components denote the position and speed along x, y, z axes. The three control inputs u_1, u_2 , and u_3 are the acceleration along x, y , and z axes, respectively.

2) *BarrierNet Design*: The robot is required to avoid a superquadratic obstacle in its path, i.e., the state of the robot should satisfy

$$(p_x - x_o)^4 + (p_y - y_o)^4 + (p_z - z_o)^4 \geq R^4 \quad (25)$$

where $(x_o, y_o, z_o) \in \mathbb{R}^3$ denotes the location of the obstacle, and $R > 0$ is the half-length of the superquadratic obstacle.

The goal is to minimize the control input effort while subject to the safety constraint (25) as the robot approaches its destination.

The relative degree of the safety constraint (25) is two with respect to the dynamics, thus, we use an HOCBF $b(\mathbf{x}) = (p_x - x_o)^4 + (p_y - y_o)^4 + (p_z - z_o)^4 - R^4$ to enforce it. Any control input $\mathbf{u}$ should satisfy the HOCBF constraint (4) which in this case (choose α_1, α_2 in Definition 4 as linear functions) is

$$-L_g L_f b(\mathbf{x}) \mathbf{u} \leq L_f^2 b(\mathbf{x}) + (p_1(z) + p_2(z)) L_f b(\mathbf{x}) + (\dot{p}_1(z) + p_1(z) p_2(z)) b(\mathbf{x}) \quad (26)$$

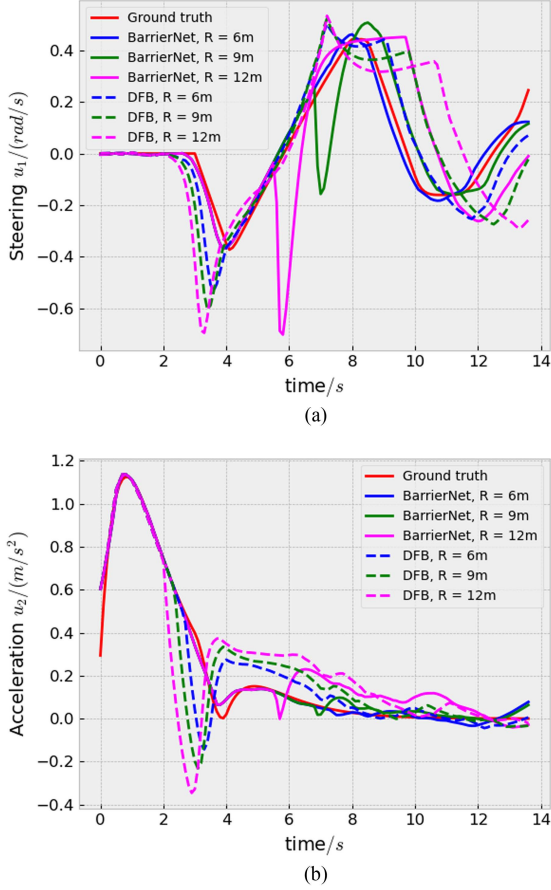


Fig. 9. Controls from the BarrierNet and DFB under different obstacle sizes. The BarrierNet and DFB are trained under the obstacle size $R = 6$ m. The results refer to the case that the trained BarrierNet/DFB controller is used to drive a robot to its destination. When we increase the obstacle size during implementation, the outputs (controls of the robot) of the BarrierNet and the DFB will adjust accordingly in order to guarantee safety, as shown by the blue and cyan curves. However, the BarrierNet tends to be less conservative for unseen situations. (a) Control u_1 . (b) Control u_2 .

where

$$\begin{aligned} L_g L_f b(\mathbf{x}) &= [4(p_x - x_o)^3, \quad 4(p_y - y_o)^3, \quad 4(p_z - z_o)^3] \\ L_f^2 b(\mathbf{x}) &= 12(p_x - x_o)^2 v_x^2 + 12(p_y - y_o)^2 v_y^2 + 12(p_z - z_o)^2 v_z^2 \\ L_f b(\mathbf{x}) &= 4(p_x - x_o)^3 v_x + 4(p_y - y_o)^3 v_y + 4(p_z - z_o)^3 v_z. \end{aligned} \quad (27)$$

In the above equations, $\mathbf{z} = \mathbf{x}$ is the input to the model, and $p_1(\mathbf{z}), p_2(\mathbf{z})$ are the trainable penalty functions. $\dot{p}_1(\mathbf{x})$ is also set as 0 as in the 2-D navigation case.

The cost in the neuron of the BarrierNet is given by

$$\min_{\mathbf{u}} (u_1 - f_1(\mathbf{z}))^2 + (u_2 - f_2(\mathbf{z}))^2 + (u_3 - f_3(\mathbf{z}))^2 \quad (28)$$

where $f_1(\mathbf{z}), f_2(\mathbf{z})$, and $f_3(\mathbf{z})$ are references controls provided by the upstream network (the outputs of the FC network).

3) *Results and Discussion:* The training data are obtained by solving a fine-tuned CBF controller introduced in [12]. We compare the FC model with our proposed BarrierNet. The training and testing results are shown in Fig. 11. The controls from the BarrierNet have some errors with respect to the ground truth,

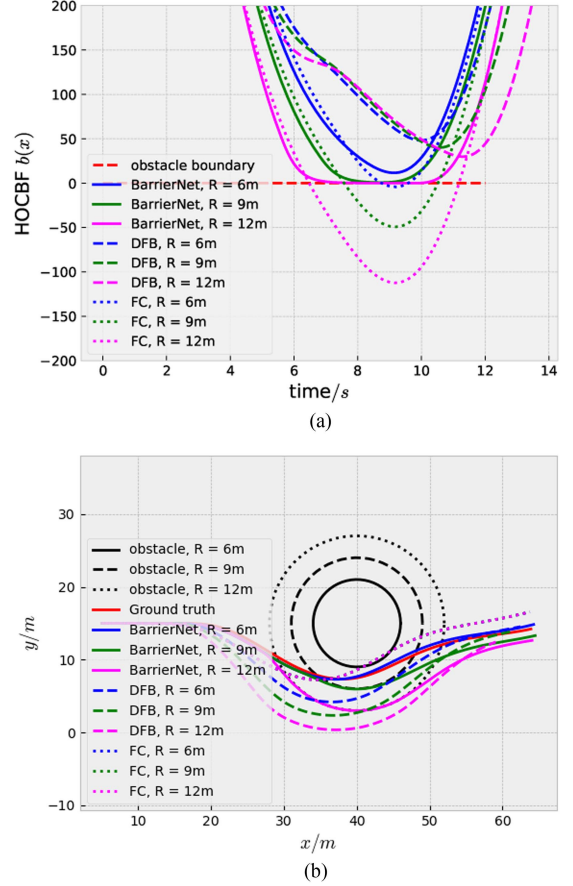


Fig. 10. Safety metrics for the BarrierNet, the DFB and the FC network. The BarrierNet, the DFB and the FC network are trained under the obstacle size $R = 6$ m. $b(\mathbf{x}) \geq 0$ implies safety guarantee. The trajectories under the FC controller coincide as the FC cannot adapt to the size change of the obstacle. (a) HOCBF $b(\mathbf{x})$ profiles under different obstacle sizes. (b) Robot trajectories under different obstacle sizes.

and this is due to the complicated safety constraint (25). We can improve the tracking accuracy with deeper BarrierNet models (not the focus of this article). Nevertheless, the implementation trajectory under the BarrierNet controller is close to the ground truth, as shown in Fig. 11(b).

The robot is guaranteed to be collision-free from the obstacle under the BarrierNet controller, as the solid-blue line shown in Fig. 11(b), while the robot from the FC may collide with the obstacle as there is no safety guarantee, as the dotted-blue line shown in Fig. 11(b). The barrier function in Fig. 11(a) also demonstrates the safety guarantees of the BarrierNet, but not in the FC model.

VII. BARRIERNET FOR VISION-BASED AUTONOMOUS DRIVING

In this section, we use the proposed BarrierNet methodology to achieve safety in a complex learning system: Vision-based end-to-end control of lane following for autonomous driving. We first discuss where the two “ends” should be defined for learning a good model based on limited data.

An end-to-end learning system is hard to interpret, especially in the setting of inferring control from high-dimensional sensor

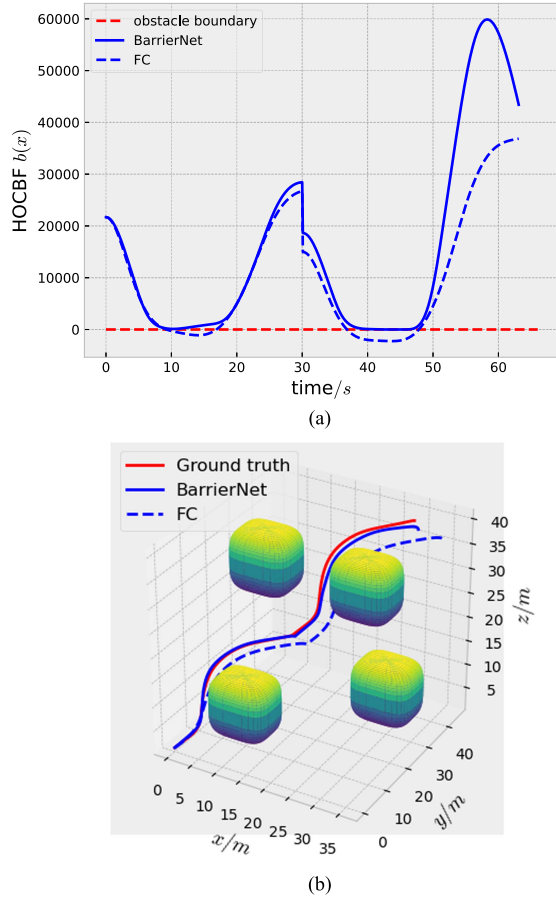


Fig. 11. HOCBFs and trajectories from the FC and BarrierNet. The results refer to the case that the trained FC/BarrierNet controller is used to drive a robot to its destination. Safety is guaranteed in the BarrierNet model but not in the FC model. (a) HOCBF $b(x)$ profiles with respect to the two approaching obstacles. (b) Robot trajectories. The initial position is at the lower-left corner.

measurements such as images. However, adding the BarrierNet layer in an end-to-end learning system provides safety guarantees.

A. Interpretable End-to-end Design

1) *System's Inputs Setup*: We note that in human driving, the majority of the information human drivers rely on comes from the front vision view. We define the input of the end-to-end architecture to be front-view images, which contain enough information for executing safe driving.

2) *System's Outputs Setup*: At the output end, where BarrierNet is implemented, high-relative-degree control variables (such as acceleration, jerk, steering rate, or steering acceleration) are generated for driving the vehicle. The first advantage of using high-relative-degree control variables is to ensure the smoothness of the vehicle states (such as speed), which results in smooth maneuvers for passenger comfort. Another advantage is to ensure the controller works with accurate maneuvering due to the physical inertia of the vehicle. If we take vehicle speed as one of the controls, and the controller requires the speed to suddenly change to a large different value, the vehicle powertrain system will fail to respond. In this case, the vehicle control becomes

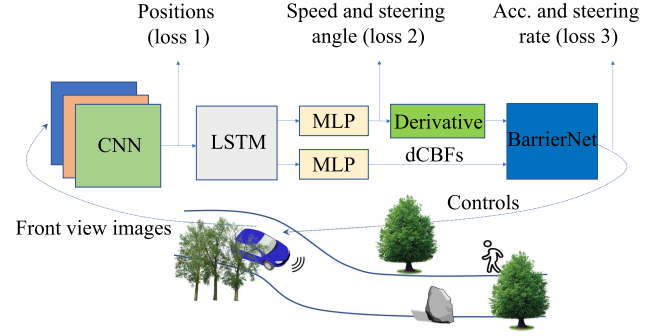


Fig. 12. Multilevel interpretable end-to-end autonomous driving framework with dCBFs. The entire pipeline is end-to-end differentiable. Each depth of the model learns different vehicle and environment information. The outputs of BarrierNet are high-relative-degree controls.

inaccurate, affecting the performance and even the safety of the vehicle.

3) *Challenges*: An end-to-end autonomous driving system with high-relative-degree control variables requires a very large and diverse training dataset. This is because more vehicle state variables are involved in higher-relative-degree controls, and thus, the vehicle may have different states under the same observation. This will cause confusion in the training process. If the training dataset is not large and diverse enough, the poor generalization of the trained controller (from open-loop to closed-loop or from sim-to-real) will make the controller fail to achieve its task. Thus, we propose a multilevel interpretable model under a limited training set, as described next.

In order to make the model interpretable, we take training loss outputs at different depths of the model. Following the convolutional neural network (CNN) or long short-term memory (LSTM), we may take part of the neurons as the loss outputs for the locations of the vehicle itself and the obstacles. This way, we train neurons at this level to learn position information. In a deeper setting, we may take part of the neurons as another loss output for the speed and steering angle of the vehicle, training these neurons to learn speed and steering angle information. By adding derivative layers following these neurons, we get the acceleration and steering rate information for the vehicle. The acceleration and steering rate could be taken as reference controls in the BarrierNet, which is also trainable, and we take the output of the BarrierNet as the final loss output in the training process. In this framework, different depths of neurons learn different vehicle information, which makes the whole structure consistent with the vehicle's physical dynamics. This architecture ensures the neural network model is interpretable. We present the model structure in Fig. 12.

B. End-to-end Policies With BarrierNet

In this subsection, we explain how we can augment BarrierNet to an end-to-end driving policy and propose solutions for challenges that arise in autonomous driving with BarrierNet.

1) *Policy Learning*: The neural policy takes in front-view images and extracts deep features based on a CNN. We then

use an **LSTM to process features for each timestamp for temporal reasoning**. The output of the LSTM (z in Definition 6) is then sent to multiple branches of multi-layer perceptrons (MLPs) to predict different parameters in BarrierNet including $H(z)$, $F(z)$, and $p_i(z)$. All parameters are then fed to the differentiable optimization layer to solve for the final controls that satisfy safety constraints enforced by dCBFs. Note that while the final controls are acceleration a and steering rate ω , we supervise the reference control $F(z)$ with velocity v and steering angle δ and compute their derivative for the reasons discussed in Section VII-A. We bound the derivative of v, δ to stabilize learning. Training objectives for the policy are mse on speed, steering angle (loss 2), cceleration, and steering rate (loss 3), as shown in Fig. 12. For training, end-to-end driving policies have been shown to achieve less robustness [53] by adopting imitation learning only (with supervision from real human-driving data). To tackle this issue, we leverage a data-driven simulation [54], [55] to augment the real human-driving dataset with more diverse data locally around the original dataset. More implementation details can be seen in Section VIII-B.

2) *State Estimation*: Besides dCBF parameters, states of the vehicle are also required in BarrierNet. Based on the notation in vehicle dynamics (29), required states include lateral displacement from the lane center d , local heading error μ , relative progress between ego-car and obstacle Δs , and displacement from lane center of the obstacle d_{obs} . We use the same CNN + LSTM architecture followed by multiple branches, with each predicting a state and mse loss as training objectives. We cap loss on $\Delta s, d_{\text{obs}}$ when the obstacle is absent or too far away to ensure states can be reasonably predicted.

3) *Safety With an Unknown Number of Constraints*: One of the challenges in autonomous driving with BarrierNet is that we have to define the exact number of the HOCBFs when designing the BarrierNet layer as it connects with previous layers. However, a vehicle may encounter a time-varying number of obstacles (constraints) in a complex environment. In order to address this problem, we proceed as follows.

Suppose $N \in \mathbb{N}$ denotes the maximum number of obstacles (such as other vehicles) a vehicle may encounter in driving. We cover each of the obstacles with an off-the-center disk, as shown in Fig. 13. The deviation direction of the disk depends on the direction of the obstacle with respect to the lane center. In this manner, we may use large disks to cover obstacles while making sure that the ego vehicle will not be overly conservative in driving through. We may use multiple small disks to cover a single obstacle. However, this increases the number of safety constraints required. Another advantage of using a large off-the-center disk is to ensure the smoothness of the vehicle trajectory and to avoid getting stuck in local traps that may appear with small disks. In this setting, we only have N safety constraints, one for each obstacle. We sort them in a specific order in connection with the previous layer and enforce them using the above differentiable HOCBFs. The order of obstacles is important since they are enforced by dCBFs, and dCBFs are connected to previous layers. We need to generate different adaptive parameters for dCBFs corresponding to obstacles whose distances are different

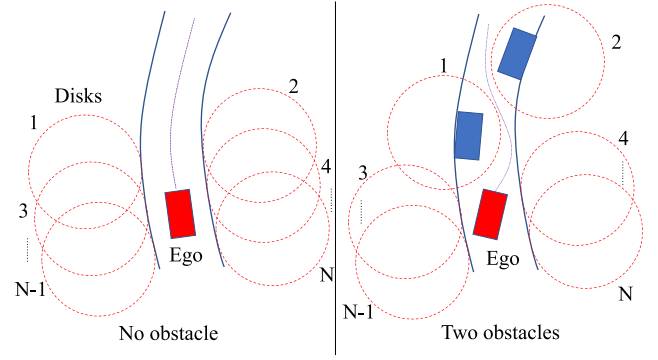


Fig. 13. Large disk covering approach for obstacle avoidance. Collisions can be avoided if the center of the vehicle never enters the disks (for a correct design of disk locations and sizes). Sorted disks are used to cover corresponding sorted obstacles as they present themselves.

Algorithm 2: Vision-Based End-to-End Autonomous Driving With BarrierNet.

Training:

- Generate road geometries with randomly placed parked vehicles for simulation purposes
- Get control labels using optimal control methods with ground truth information
- Obtain front-view images using the generated vehicle trajectories from optimal control
- Pair optimal controls and front-view images as training set
- Train the BarrierNet with the data set using Alg. 1

Testing:

while not end do

- Estimate ego vehicle state
- Evaluate the number of obstacles and their corresponding states
- Get the current front-view image
- Forward propagate the model with the estimated states and front-view image to get control
- Apply the control to the ego vehicle

end

with respect to the ego vehicle. Thus, the connection to the corresponding dCBFs (i.e., the order of obstacles) is crucial.

When there are no actual obstacles on the road, as in the case depicted on the left-hand side of Fig. 13, we just move the covering disks off the road. It is important to note that we have additional lane-keeping constraints [such as (30)], and thus, these disks are not used to approximate the lane boundaries. Instead, they are placed in a way such that the road is not blocked. These disks move along the road as the vehicle progresses at the same speed. While the vehicle drives on the road, these disks do not affect its motion as the corresponding HOCBF constraints are not activated.

When there are one or more obstacles on the road, as in the case depicted on the right-hand side of Fig. 13, we first sort the obstacles according to their distance with respect to the ego vehicle. Then, we use the sorted disks to cover the corresponding

sorted obstacles. The sorted covering approach can make sure that the vehicle may leave the road in order to avoid collision with obstacles. In this setting, although we may have redundant differentiable HOCBFs in terms of obstacle avoidance, these HOCBFs always play an important role in guiding the vehicle, either in lane keeping or obstacle avoidance.

The whole process for vision-based end-to-end autonomous driving includes the following:

- 1) generating training data and control labels;
- 2) training the model using supervised learning;
- 3) estimating the vehicle state and obstacle state in real time during testing;
- 4) forward propagation of the model with front-view images as inputs to generate safe controls for the ego vehicle.

We summarize the algorithm for end-to-end autonomous driving in Algorithm 2.

4) *Multiple and Active Obstacles*: The BarrierNet can handle multiple obstacles by adding the corresponding dCBF constraints in the differentiable QP. Each of the two road boundaries could be viewed as an obstacle in autonomous driving. The BarrierNet can also work for dynamic obstacles (such as other active vehicles, pedestrians, etc.), and the safety guarantees of dynamic obstacles in the BarrierNet require additional state estimations (such as moving speed) for those obstacles. In fact, image-based state estimations are usually hard for states other than positions (such as speed, acceleration, etc.). Thus, we may need to use other sensors (such as light detection and ranging) to better estimate states for dynamic obstacles and conduct multisensor fusion. The uncertainty of surrounding agents can be considered in the dCBFs using the uncertainty bound if it is known, as shown in [52]. The conservativeness of this approach can be addressed using the adaptive event-driven approach for CBFs [56].

VIII. VISION-BASED AUTONOMOUS DRIVING EXPERIMENTS

In this section, we show experiments with the proposed vision-based end-to-end autonomous driving framework in both sim-to-real environments and a full-scale autonomous vehicle. We start by introducing the hardware platform and data collection, followed by implementation details of the proposed model. We then demonstrate extensive analysis in the sim-to-real environment virtual image synthesis and transformation for autonomy (VISTA) [55]. Finally, we showcase results with real-car deployment.

A. Hardware Setup and Real-World Data Collection

We deploy our models onboard a full-scale autonomous vehicle (2019 Lexus RX 450H) equipped with an NVIDIA 2080Ti GPU and an AMD Ryzen 7 3800X 8-Core Processor. We use an red, green and blue (RGB) camera BFS-PGE-23S3C-CS as the primary perception sensor, which runs at 30 Hz, with a resolution of 960×600 , and has 130° horizontal field-of-view. Other onboard sensors include inertial measurement sensors and wheel encoders to measure steering feedback and odometry. Also, we use a differential global positioning system (dGPS) for evaluation purposes. To run the data-driven simulation VISTA [55],

we collect real-world data from a wide range of environments, including different times of day, weather conditions, and seasons of the year. The entire dataset consists of roughly 2 h of driving data, which is further augmented with our training dataset generation pipeline using VISTA.

B. Synthetic Training Dataset Generation

We train our model with guided policy learning, which has been shown to improve effectiveness for direct model transfer to real-car deployment (other techniques, such as conformance checking [27], can also be used to achieve sim-to-real transfer). The data generation process follows: 1) in VISTA, randomly initializing both ego- and ado-car with different configurations like relative poses, geographical locations associated with the real dataset, and the appearance of the vehicle; 2) running an optimal controller with access to privileged information to steer the ego-vehicle and collect ground-truth control outputs with corresponding states; 3) collecting RGB images at viewpoints along the trajectories. We choose nonlinear model predictive control (NMPC) as the privileged (nominal) controller. While NMPC is usually computationally expensive and hard to solve, it is tractable offline and, with jerk u_{jerk} and steering acceleration u_{steer} as controls, provides smooth acceleration a and steering rate w , which is used as learning targets in BarrierNet. Vehicle dynamics of NMPC and BarrierNet (1) are defined with respect to a reference trajectory [57]. It measures the along-trajectory distance $s \in \mathbb{R}$ and the lateral distance $d \in \mathbb{R}$ of the vehicle center of gravity (CoG) with respect to the closest point on the reference trajectory

$$\underbrace{\begin{bmatrix} \dot{s} \\ \dot{d} \\ \dot{\mu} \\ \dot{v} \\ \dot{\delta} \\ \dot{\omega} \end{bmatrix}}_{\dot{x}} = \underbrace{\begin{bmatrix} \frac{v \cos(\mu + \beta)}{1 - d\kappa} \\ v \sin(\mu + \beta) \\ \frac{v}{l_r} \sin \beta - \kappa \frac{v \cos(\mu + \beta)}{1 - d\kappa} \\ a \\ 0 \\ \omega \\ 0 \end{bmatrix}}_{f(x)} + \underbrace{\begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}}_{g(x)} \underbrace{\begin{bmatrix} u_{\text{jerk}} \\ u_{\text{steer}} \end{bmatrix}}_u \quad (29)$$

where μ is the vehicle local heading error determined by the difference of the global vehicle heading $\theta \in \mathbb{R}$ and the tangent angle $\phi \in \mathbb{R}$ of the closest point on the reference trajectory (i.e., $\theta = \phi + \mu$) as shown in Fig. 14; v , a denote the vehicle linear speed and acceleration, respectively; δ , ω denote the steering angle and steering rate, respectively; κ is the curvature of the reference trajectory at the closest point; l_r is the length of the vehicle from the tail to the CoG; and u_{jerk} , u_{steer} denote the two control inputs for jerk and steering acceleration (in the nominal controller), respectively. $\beta = \arctan(\frac{l_r}{l_r + l_f} \tan \delta)$, where l_f is the length of the vehicle from the head to the CoG. We set the receding horizon of the NMPC to 20-time steps during data sampling, and it is implemented in a virtual simulation environment in MATLAB. We augment the real-world dataset using VISTA and NMPC with synthetic obstacle avoidance and lane following data. In total, the training dataset has around 400 k images.

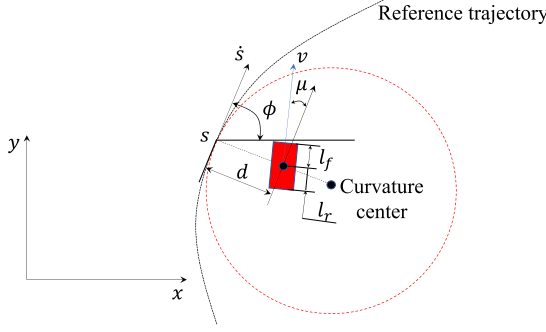


Fig. 14. Coordinates of ego w.r.t a reference trajectory.

BarrierNet Design: The vehicle is required to stay within the lane and avoid obstacles along its path, i.e., the state of the vehicle should satisfy

$$\begin{aligned} |d| &\leq d_{lf} \\ \Delta s^2 + (d - d_{obs})^2 &\geq r_D^2 \end{aligned} \quad (30)$$

where d_{lf} is a preset bound of how close the car needs to stay close to the lane center. If the road width changes, we can replace d_{lf} by a variable that is either known or predicted from the image observation. Δs is relative progress between ego-car and obstacle, r_D is disk size, and d_m is the lateral displacement from lane center of the obstacle.

The goal is to minimize the control input effort while subject to the safety constraint (30) as the robot approaches its destination.

The relative degree of the safety constraints (30) is two with respect to the dynamics, thus, we use three HOCBFs $b_{lf}^{\text{left}} = d_{lf} - d$, $b_{lf}^{\text{right}} = d_{lf} + d$, and $b_{obs} = \Delta s^2 + (d - d_{obs})^2 - r_D^2$ to enforce them. Any control input u should satisfy the HOCBF constraint (4) which in this case (choose α_1, α_2 in Definition 4 as linear functions) is

$$\begin{aligned} -L_g L_f b(x) u &\leq L_f^2 b(x) + (p_1(z) + p_2(z)) L_f b(x) \\ &+ (\dot{p}_1(z) + p_1(z) p_2(z)) b(x) \end{aligned} \quad (31)$$

where

$$\begin{aligned} L_g L_f b_{lf}^{\text{left}}(x) &= \left[\frac{-s_{\mu,\beta}}{\delta^2(1+(l_{rf}+\delta)^2)} \right] \\ L_f^2 b_{lf}^{\text{left}}(x) &= -vc_{\mu,\beta} \dot{\mu} \\ L_f b_{lf}^{\text{left}}(x) &= -vs_{\mu,\beta} \end{aligned} \quad (32)$$

$$\begin{aligned} L_g L_f b_{lf}^{\text{right}}(x) &= \left[\frac{s_{\mu,\beta}}{\delta^2(1+(l_{rf}+\delta)^2)} \right] \\ L_f^2 b_{lf}^{\text{right}}(x) &= vc_{\mu,\beta} \dot{\mu} \\ L_f b_{lf}^{\text{right}}(x) &= vs_{\mu,\beta} \end{aligned} \quad (33)$$

$$L_g L_f b_{obs}(x) = \left[\frac{2\Delta s c_{\mu+\beta}}{1-d\kappa} + 2(d-d_{obs})s_{\mu+\beta} \right] - \frac{2\Delta s v s_{\mu+\beta}}{1-d\kappa} + \frac{2(d-d_{obs})vc_{\mu+\beta}l_{rf}}{\delta^2(1+l_{rf}\delta)^2}$$

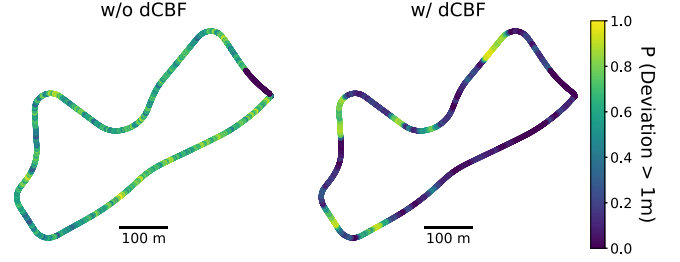


Fig. 15. Lane following probabilistic comparisons of deviation from the lane center in a BarrierNet with/without lane keeping CBFs.

$$\begin{aligned} L_f^2 b_{obs}(x) &= 2 \left(\frac{vc_{\mu,\beta}}{1-d\kappa} \right)^2 + 2(vs_{\mu,\beta})^2 - \frac{2\Delta s v s_{\mu,\beta} \dot{\mu}}{1-d\kappa} \\ &\times \frac{2\Delta s \kappa v^2 s_{\mu,\beta} c_{\mu,\beta}}{(1-d\kappa)^2} + 2(d-d_{obs})vc_{\mu,\beta} \dot{\mu} \\ L_f b_m(x) &= \frac{2\Delta s v c_{\mu,\beta}}{1-d\kappa} + 2(d-d_{obs})vs_{\mu,\beta} \end{aligned} \quad (34)$$

where $s_{\mu,\beta} = \sin(\mu + \beta)$, $c_{\mu,\beta} = \cos(\mu + \beta)$, $l_{rf} = \frac{l_r + l_f}{2}$, $\Delta s = s - s_{obs}$, $\dot{\mu} = \frac{v \sin(\beta)}{l_r} - \frac{\kappa v c_{\mu,\beta}}{1-d\kappa}$.

In the above equations, z is the deep feature extracted from the front-view images, and $p_1(z)$ and $p_2(z)$ are the trainable penalty functions. $\dot{p}_1(x)$ is also set as 0 as in the 2-D navigation case.

The cost in the neuron of the BarrierNet is given by

$$\min_u (u_1 - f_1(z))^2 + (u_2 - f_2(z))^2 \quad (35)$$

where $f_1(z)$ and $f_2(z)$ are references controls provided by the upstream network (the outputs of the CNN + LSTM).

C. Evaluation in Sim-to-Real Environments

Open-loop control error (i.e., the difference between predicted and ground-truth control) has been shown to be a poor indicator to evaluate the performance of a driving policy since it only measures error around ground-truth trajectories and ignores accumulated errors that gradually drift the vehicle to out-of-distribution regions. Hereby, we present closed-loop testing results in the sim-to-real environment VISTA [55].

1) Lane Keeping as Safety Constraints: In Fig. 15, we show the probability of ego-vehicle deviating away from the lane center for larger than 1 m. We run 1000 episodes with maximum 200 steps if not crashed (off-lane more than 2 m) prematurely. In each episode, the vehicle is randomly initialized at a point in the trace, and we compute the average deviation at every point to ensure a sufficiently large sample size for the statistics. The model with lane-keeping CBFs achieves significantly better performance since they can encourage the autonomous vehicle to stay close to the lane center by decreasing the boundary values due to the Lyapunov property of CBFs.

2) Obstacle Avoidance: In Table III, we show the crash rate and minimal clearance of models with or without BarrierNet and with or without access to ground-truth states. Minimal clearance is computed as the closest distance between polygons

TABLE III
CRASH RATE AND CLEARANCE WITH/WITHOUT BARRIERNET, USING OR NOT
USING GROUND TRUTH OBSTACLE INFORMATION

Method	Crash rate ↓	Min. clearance (m) ↑
w/o dCBF	0.53	0.43
w/ dCBF	0.28	0.55
w/ dCBF (with gt)	0.03	0.61

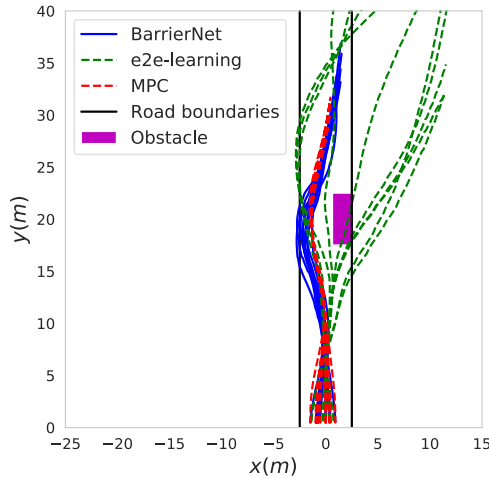


Fig. 16. Vehicle trajectories in obstacle avoidance using MPC, end-to-end learning (e2e-learning), and the proposed BarrierNet in VISTA. The MPC method employs accurate ground truth vehicle state and obstacle information, while e2e-learning and BarrierNet do not have such information and infer it from images. The e2e-learning method fails to guarantee safety.

of ego- and ado-car within an episode. The introduction of obstacle avoidance dCBFs significantly reduces the crash rate and increases clearance. The remaining failures mainly come from the imprecise or even erroneous state and obstacle information inferred from the front-view camera only. With access to ground-truth information (an ideal state estimator), the crash rate is close to yet not zero. This might be due to misaligned dynamics and inter-sampling effects of CBFs, which have been extensively studied in CBFs [5], [32].

3) *BarrierNet Provides Safe Maneuvers*: In Fig. 16, we benchmarked for different learning systems, end-to-end learning (e2e-learning) and BarrierNet, to provide driving trajectories under the same configuration except for arbitrary initial pose on the road. We show ten variants of initial states for each model. We also compare the proposed BarrierNet with model predictive control (MPC). The MPC method employs accurate ground truth vehicle state and obstacle information, and thus, the resulting trajectories are consistent. It can be observed that trajectories from the three models mostly align with each other in the beginning as the ego-vehicle starts with different lateral displacements from the lane center and tries to recover. Then, the three sets of trajectories diverge while approaching the obstacle. This is the consequence of correction from the activated dCBFs over the reference unsafe control. With BarrierNet, safety is guaranteed. The trajectories from the BarrierNet are not as consistent as (but are still close to) the MPC ones since the

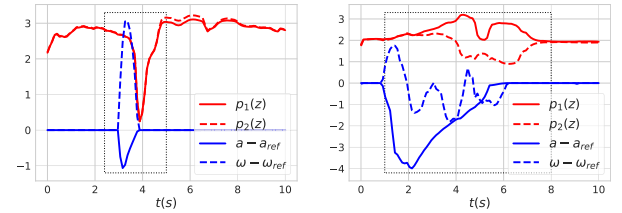


Fig. 17. Penalty $p_1(z)$, $p_2(z)$ variation in a dCBF (BarrierNet) when approaching an obstacle under two (different) trained BarrierNets. The relative degree of the safety constraint is two, and thus we have two CBF parameters in one CBF. The segments inside the dotted boxes denote intervals when the ego vehicle is near the obstacle. The box sizes are different as the ego has different speeds when passing the obstacle.

BarrierNet infers vehicle state and obstacle information from images that induce uncertainties. On the other hand, the MPC is very computationally expensive (MPC computation time 0.872 s versus BarrierNet computation time 0.004 s on the same machine). Although linearization is possible in MPC to make it more efficient, this may make it lose guarantees. The computation times of BarrierNet and e2e-learning are close.

4) *BarrierNet With Different Profiles*: We also notice that the BarrierNet may learn different CBF parameters when the ego vehicle approaches an obstacle. In Fig. 17, we present two possible variations of penalty functions $p_1(z)$ and $p_2(z)$ when the ego vehicle is around an obstacle. The penalty functions $p_1(z)$ and $p_2(z)$ adapt to the obstacle when the ego vehicle is close to an obstacle, and they recover to some values when the ego leaves the obstacle. This shows the flexibility of the BarrierNet. Another observation is that the outputs of the BarrierNet tend to deviate from the reference controls (from the previous LSTM layer) when the ego vehicle is close to the obstacle. This shows the safety guarantee property of CBFs. In order to avoid this deviation, we need to improve the learned model with better reference controls and CBF parameters.

D. Physical Autonomous Car Experiments

To verify the effectiveness of the proposed vision-based end-to-end framework with dCBFs, we deploy the trained models on a full-scale autonomous driving car. The experiments are conducted in a test site with a rural road type. We majorly test the algorithm with AR and only perform the minimal experiment with real-car obstacles for safety reasons. We use a precollected map of the test site and vehicle poses from the dGPS to place virtual obstacles in the front of the ego-vehicle on the road with AR. Note that the tested models are still using vision inputs only to steer the autonomous vehicle without any access to the ground-truth state. Fig. 18 is an illustration of the real-car experimental setup. Another thing worth mentioning is that the scene is covered with snow at the time we conducted real car experiments. The icy road surface at the track and heavy snow at the side of the road introduce tire slippage and pose additional challenges to our self-driving system. Also, the reflection of sunlight on the ice makes it hard to recognize road boundaries, even from human judgment. With high-precision dGPS in the

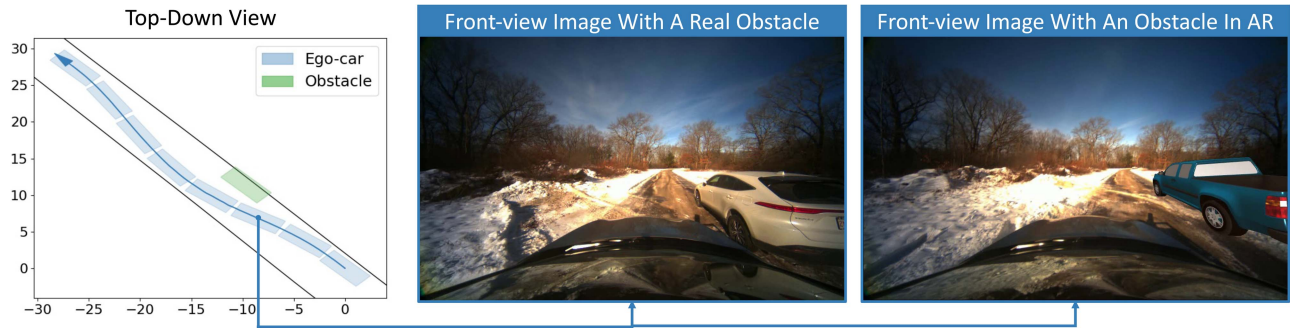


Fig. 18. Illustration of real-car experiments. We show a driver's view with a real car as obstacle in front of it (middle) and with a car in front of it using AR (right).

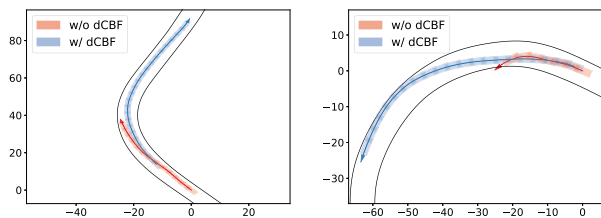


Fig. 19. Two cases of experimental vehicle trajectories in lane keeping with/without lane keeping CBFs in the BarrierNet. Tire slipping happened on the icy road.

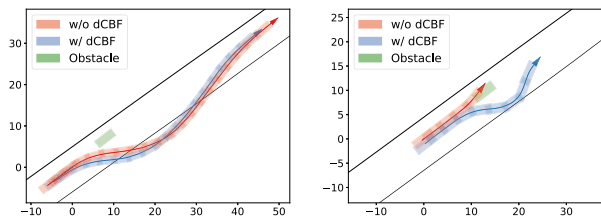


Fig. 20. Two cases of experimental vehicle trajectories in obstacle avoidance with/without BarrierNet. In the left case, the heavy snow by the road is preventing the vehicle from getting back to the road due to tire slipping, and thus the vehicle recovers slowly even when the steering wheel is at its left limit.

site (covariance < 1 cm), we provide qualitative analysis with a side-by-side comparison between models with and without BarrierNet.

1) *BarrierNet in Challenging Sharp Turns*: In Fig. 19, we demonstrate the driving trajectories of BarrierNet with and without lane-keeping CBFs in sharp left and right turns. We show the footprint of the vehicle through time and indicate the forward direction with arrows. Without lane-keeping CBFs (red), the car is more prone to get off-road, while roughly correct estimates of deviation from lane center (d) imposes an additional layer of safety with lane-keeping CBFs (blue).

2) *Obstacle Avoidance in Real World*: We also did experiments on the autonomous car in obstacle avoidance, as shown in Fig. 20. The first example (left) demonstrates that with reasonable reference control (both models successfully avoid the obstacle), the model with obstacle avoidance dCBFs (blue) creates more clearance to achieve better safety. The second example (right) highlights the effectiveness of BarrierNet (blue)

when the reference control (red) fails to avoid the front car and requires correction from activated dCBF constraints.

3) *Limitations*: The proposed BarrierNet is subject to several challenges in vision-based driving, which motivates further future work as follows: 1) CBFs for all kinds of traffic participants are constructed using a disk covering approach. However, how to efficiently construct CBFs is still challenging (including CBFs for lane keeping) when there are many participants; 2) occluded obstacles introduce uncertainties to the safety of the ego vehicle; 3) safety guarantees under dynamic obstacles are challenging since we also need to know obstacle dynamics and states.

IX. CONCLUSION

In this article, we proposed BarrierNet - a differentiable HOCBF layer that is trainable and guarantees safety with respect to the user-defined safe sets. BarrierNet can be integrated with any upstream neural network controller to provide a safety layer. In our experiments, we show that the proposed BarrierNet can guarantee safety while addressing the conservativeness that CBFs induce. A potential future avenue of research emerging from this work will be to simultaneously learn the system dynamics and unsafe sets with BarrierNets. This can be enabled using the expressive class of continuous-time neural network models [58], [59], [60].

ACKNOWLEDGMENT

The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the United States Air Force or the USA government. The U.S. Government is authorized to reproduce and distribute reprints for government purposes, notwithstanding any copyright notation herein.

REFERENCES

- [1] A. D. Ames, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs with application to adaptive cruise control," in *Proc. IEEE 53rd Conf. Decis. Control*, 2014, pp. 6271–6278.
- [2] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, Aug. 2017.

- [3] Q. Nguyen and K. Sreenath, "Exponential control barrier functions for enforcing high relative-degree safety-critical constraints," in *Proc. Amer. Control Conf.*, 2016, pp. 322–328.
- [4] L. Wang, E. A. Theodorou, and M. Egerstedt, "Safe learning of quadrotor dynamics using barrier certificates," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2018, pp. 2460–2465.
- [5] A. Taylor, A. Singletary, Y. Yue, and A. Ames, "Learning for safety-critical control with control barrier functions," in *Proc. Learn. Dyn. Control*, 2020, pp. 708–717.
- [6] J. Choi, F. Castañeda, C. J. Tomlin, and K. Sreenath, "Reinforcement learning for safety-critical control under model uncertainty, using control Lyapunov functions and control barrier functions," in *Proc. Robot.: Sci. Syst.*, 2020.
- [7] A. J. Taylor, A. Singletary, Y. Yue, and A. D. Ames, "A control barrier perspective on episodic learning via projection-to-state safety," *IEEE Contr. Syst. Lett.*, vol. 5, no. 3, pp. 1019–1024, Jul. 2021.
- [8] X. Xu, P. Tabuada, J. W. Grizzle, and A. D. Ames, "Robustness of control barrier functions for safety critical control," *IFAC-PapersOnLine*, vol. 48, no. 27, pp. 54–61, 2015.
- [9] T. Gurriet, A. Singletary, J. Reher, L. Ciarletta, E. Feron, and A. Ames, "Towards a framework for realizable safety critical control through active set invariance," in *Proc. ACM/IEEE 9th Int. Conf. Cyber-Phys. Syst.*, 2018, pp. 98–106.
- [10] A. J. Taylor and A. D. Ames, "Adaptive safety with control barrier functions," in *Proc. Amer. Control Conf.*, 2020, pp. 1399–1405.
- [11] N. Csomay-Shanklin, R. K. Cosner, M. Dai, A. J. Taylor, and A. D. Ames, "Episodic learning for safe bipedal locomotion with control barrier functions and projection-to-state safety," in *Proc. Learn. Dyn. Control*, 2021, pp. 1041–1053.
- [12] W. Xiao and C. Belta, "Control barrier functions for systems with high relative degree," in *Proc. IEEE 58th Conf. Decis. Control*, 2019, pp. 474–479.
- [13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [14] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [15] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [16] M. Lechner and R. Hasani, "Learning long-term dependencies in irregularly-sampled time series," 2020, *arXiv:2006.04418*.
- [17] R. Hasani et al., "Closed-form continuous-time neural networks," *Nature Mach. Intell.*, Nature Publishing Group UK London, 2022, pp. 1–12.
- [18] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [19] A. E. Bryson and Y.-C. Ho, *Applied Optimal Control*. Waltham, MA, USA: Ginn Blaisdell, 1969.
- [20] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Madison, WI, USA: Nob Hill Publishing, 2017.
- [21] S. Mitsch, K. Ghorbal, and A. Platzer, "On provably safe obstacle avoidance for autonomous robotic ground vehicles," in *Proc. Robot.: Sci. Syst.*, 2013.
- [22] C. Pek, S. Manzingier, M. Koschi, and M. Althoff, "Using online verification to prevent autonomous vehicles from causing accidents," *Nature Mach. Intell.*, vol. 2, no. 9, pp. 518–528, 2020.
- [23] M. Althoff and J. M. Dolan, "Online verification of automated road vehicles using reachability analysis," *IEEE Trans. Robot.*, vol. 30, no. 4, pp. 903–918, Aug. 2014.
- [24] S. M. LaValle, J. Kuffner, and J. James, "Randomized kinodynamic planning," *Int. J. Robot. Res.*, vol. 20, no. 5, pp. 378–400, 2001.
- [25] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, Jul. 1968.
- [26] K. Ota et al., "Deep reactive planning in dynamic environments," in *Proc. Conf. Robot Learn.*, 2021, pp. 1943–1957.
- [27] H. Roehm, J. Oehlerking, M. Woehrle, and M. Althoff, "Model conformance for cyber-physical systems: A survey," *ACM Trans. Cyber-Phys. Syst.*, vol. 3, no. 3, pp. 1–26, 2019.
- [28] G. Yang, C. Belta, and R. Tron, "Self-triggered control for safety critical systems using control barrier functions," in *Proc. Amer. Control Conf.*, 2019, pp. 4454–4459.
- [29] W. Xiao, C. Belta, and C. G. Cassandras, "Event-triggered safety-critical control for systems with unknown dynamics," in *Proc. IEEE 60th Conf. Decis. Control*, 2021, pp. 540–545.
- [30] J. Breeden and D. Panagou, "High relative degree control barrier functions under input constraints," in *Proc. IEEE 60th Conf. Decis. Control*, 2021, pp. 6119–6124.
- [31] W. Xiao, C. Belta, and C. G. Cassandras, "Sufficient conditions for feasibility of optimal control problems using control barrier functions," *Automatica*, vol. 135, 2022, Art. no. 109960.
- [32] W. Xiao, C. Belta, and C. G. Cassandras, "Adaptive control barrier functions," in *IEEE Trans. Autom. Control*, vol. 67, no. 5, pp. 2267–2281, May 2022, doi: 10.1109/TAC.2021.3074895.
- [33] A. Robey et al., "Learning control barrier functions from expert demonstrations," in *Proc. IEEE 59th Conf. Decis. Control*, 2020, pp. 3717–3724.
- [34] M. Srinivasan, A. Dabholkar, S. Coogan, and P. A. Vela, "Synthesis of control barrier functions using a supervised machine learning approach," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 7139–7145.
- [35] B. T. Lopez, J. J. E. Slotine, and J. P. How, "Robust adaptive control barrier functions: An adaptive and data-driven approach to safety," *IEEE Contr. Syst. Lett.*, vol. 5, no. 3, pp. 1031–1036, Jul. 2021.
- [36] S. Yaghoubi, G. Fainekos, and S. Sankaranarayanan, "Training neural network controllers using control barrier functions in the presence of disturbances," in *Proc. IEEE 23rd Int. Conf. Intell. Transp. Syst.*, 2020, pp. 1–6.
- [37] M. A. Pereira, Z. Wang, I. Exarchos, and E. A. Theodorou, "Safe optimal control using stochastic barrier functions and deep forward-backward SDEs," in *Proc. Conf. Robot Learn.*, 2020, pp. 1783–1801.
- [38] B. Amos and J. Z. Kolter, "Optnet: Differentiable optimization as a layer in neural networks," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, vol. 70, pp. 136–145.
- [39] B. Amos, I. D. J. Rodriguez, J. Sacks, B. Boots, and J. Z. Kolter, "Differentiable MPC for end-to-end planning and control," in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 8299–8310.
- [40] P.-F. Massiani, S. Heim, and S. Trimpe, "On exploration requirements for learning safety constraints," in *Proc. Learn. Dyn. Control*, 2021, pp. 905–916.
- [41] S. Gruenbacher et al., "GoTube: Scalable stochastic verification of continuous-depth models," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 6, 2022, pp. 6755–6764.
- [42] S. Grunbacher, R. Hasani, M. Lechner, J. Cyranka, S. A. Smolka, and R. Grosu, "On the verification of neural odes with stochastic guarantees," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, pp. 11525–11535.
- [43] J. V. Deshmukh, J. P. Kapinski, T. Yamaguchi, and D. Prokhorov, "Learning deep neural network controllers for dynamical systems with safety guarantees: Invited paper," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2019, pp. 1–7.
- [44] J. Ferlez, M. Elnaggar, Y. Shoukry, and C. Fleming, "ShieldDNN: A provably safe NN filter for unsafe NN controllers," 2020, *arXiv:2006.09564*.
- [45] W. Jin, Z. Wang, Z. Yang, and S. Mou, "Neural certificates for safe control policies," 2020, *arXiv:2006.08465*.
- [46] H. Zhao, X. Zeng, T. Chen, and J. Woodcock, "Learning safe neural network controllers with barrier certificates," *Form Asp Comput.*, vol. 33, pp. 437–455, 2021.
- [47] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 2002.
- [48] W. Xiao, C. G. Cassandras, C. A. Belta, and D. Rus, "Control barrier functions for systems with multiple control inputs," in *Proc. Amer. Control Conf.*, 2022, pp. 2221–2226.
- [49] W. Xiao et al., "Rule-based optimal control for autonomous driving," in *Proc. ACM/IEEE 12th Int. Conf. Cyber-Phys. Syst.*, 2021, pp. 143–154.
- [50] Y. Ye and E. Tse, "An extension of Karmarkar's projective algorithm for convex quadratic programming," *Math. Program.*, vol. 44, no. 1, pp. 157–179, 1989.
- [51] W. Xiao and C. G. Cassandras, "Decentralized optimal merging control for connected and automated vehicles with safety constraint guarantees," *Automatica*, vol. 123, 2021, Art. no. 109333.
- [52] W. Xiao, C. G. Cassandras, and C. Belta, "Bridging the gap between optimal trajectory planning and safety-critical control with applications to autonomous vehicles," *Automatica*, vol. 129, 2021, Art. no. 109592.
- [53] A. Amini et al., "Learning robust control policies for end-to-end autonomous driving from data-driven simulation," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 1143–1150, Apr. 2020.
- [54] T.-H. Wang, A. Amini, W. Schwarting, I. Gilitschenski, S. Karaman, and D. Rus, "Learning interactive driving policies via data-driven simulation," in *Proc. Int. Conf. Robot. Automat.*, 2022, pp. 7745–7752.

- [55] A. Amini et al., “Vista 2.0: An open, data-driven simulator for multimodal sensing and policy learning for autonomous vehicles,” in *Proc. Int. Conf. Robot. Automat.*, 2022, pp. 2419–2426.
- [56] W. Xiao, C. Belta, and C. G. Cassandras, “Event-triggered control for safety-critical systems with unknown dynamics,” *IEEE Trans. Autom. Control*, 2022, early access, doi: [10.1109/TAC.2022.3202088](https://doi.org/10.1109/TAC.2022.3202088).
- [57] A. Rucco, G. Notarstefano, and J. Hauser, “An efficient minimum-time trajectory generation strategy for two-track car vehicles,” *IEEE Trans. Control Syst. Technol.*, vol. 23, no. 4, pp. 1505–1519, Jul. 2015.
- [58] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, “Neural ordinary differential equations,” in *Proc. 32nd Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 6572–6583.
- [59] R. Hasani, M. Lechner, A. Amini, D. Rus, and R. Grosu, “Liquid time-constant networks,” in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, pp. 7657–7666.
- [60] C. Vorbach, R. Hasani, A. Amini, M. Lechner, and D. Rus, “Causal navigation by continuous-time neural networks,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 12425–12440.



Wei Xiao (Member, IEEE) received the B.Sc. degree in mechanical engineering and automation from the University of Science and Technology Beijing, Beijing, China, the M.Sc. degree in robotics from the Chinese Academy of Sciences (Institute of Automation), Beijing, China, and the Ph.D. degree in systems engineering from Boston University, Boston, MA, USA, in 2013, 2016, and 2021, respectively.

He is currently a Postdoctoral Associate with the Massachusetts Institute of Technology, Cambridge, MA, USA. His current research interests include control theory and machine learning, with particular emphasis on robotics and traffic control.

Dr. Xiao was the recipient of an Outstanding Student Paper Award at the 2020 IEEE Conference on Decision and Control.



Makram Chahine received the Diplôme d’Ingénieur in applied mathematics from École Centrale Paris, Rennes, France, and the M.Sc. degree in aerospace engineering from the Georgia Institute of Technology, Atlanta, GA, USA, both in 2019. Since 2021, he has been working toward the Ph.D. degree in electrical engineering and computer science with the Massachusetts Institute of Technology, Cambridge, MA, USA.

His research interests include autonomous robots, artificial intelligence, and complex interacting systems.



Alexander Amini (Member, IEEE) received the B.S. and M.S. Ph.D. degrees from the Massachusetts Institute of Technology (MIT), Cambridge, MA, USA, in 2017, 2018, and 2022, respectively, both in electrical engineering and computer science, with a minor in mathematics.

He is a Postdoctoral Research Associate with the MIT, in the Computer Science and Artificial Intelligence Laboratory (CSAIL), with Prof. Daniela Rus. His research interests include developing the science and engineering of autonomy and its applications to

safe decision making for autonomous agents. His work has spanned learning end-to-end control (i.e., perception-to-actuation) of autonomous systems, formulating confidence of neural networks, mathematical modeling of human mobility, as well as building complex inertial refinement systems.

Mr. Amini is a recipient of the NSF Graduate Research Fellowship.



Tsun-Hsuan Wang (Student Member, IEEE) received the B.Sc. and M.Sc. degrees from National Tsing Hua University, Hsinchu City, Taiwan, in 2017 and 2020, respectively, both in electrical engineering. He received the Ph.D. degree in electrical engineering and computer science from the Computer Science and Artificial Intelligence Laboratory (CSAIL), Massachusetts Institute of Technology (MIT), Cambridge, MA, USA.

His research interests include simulation for robot learning, intersection between control theory and machine learning, and structures and explainability in neural policies, on the application of autonomous driving, flight control, and soft robotics.



Xiao Li (Member, IEEE) received the Ph.D. degree in mechanical engineering from Boston University, Boston, MA, USA, in 2019.

He is currently a Postdoctoral Associate with the Massachusetts Institute of Technology (MIT), Cambridge, MA, USA Computer Science and Artificial Intelligence Lab (CSAIL). His research interests include reinforcement learning, trajectory prediction, and generation with applications in robotic manipulation and autonomous driving.



Ramin Hasani received the Ph.D. degree with distinction in computer science from the Vienna University of Technology (TU Wien), Vienna, Austria, in 2020.

He was a Postdoctoral Associate with the Computer Science and Artificial Intelligence Lab (CSAIL), Massachusetts Institute of Technology (MIT), Cambridge, MA, USA, leading research on modeling intelligence and sequential decision-making, with Prof. Daniela Rus. He is currently a Principal AI and Machine Learning Scientist with the

Vanguard Group and a Research Affiliate with the CSAIL, MIT. His research interests include robust deep learning and decision-making in complex dynamical systems.

Dr. Hasani was the recipient of the HPC Innovation Excellence Award in 2022 and was nominated for the TÜV Austria Dissertation Award in 2020 for his Ph.D. dissertation and continued research on Liquid Neural Networks, which got recognized internationally.



Daniela Rus (Fellow, IEEE) received the Ph.D. degree in computer science from Cornell University, Ithaca, NY, USA, in 1993.

She was a Professor with the Department of Computer Science, Dartmouth College, Hanover, NH, USA. She is currently the Andrew (1956) and Erna Viterbi Professor of Electrical Engineering and Computer Science and the Director of the Computer Science and Artificial Intelligence Laboratory with the Massachusetts Institute of Technology (MIT), Cambridge, MA, USA. Her research interests include

robotics, mobile computing, and big data. The key focus of her research is to develop the science of networked/distributed/collaborative robotics.

Dr. Rus is a Class of 2002 MacArthur Fellow, a Fellow of ACM and AAAI, and a Member of the National Academy of Engineering.